

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación



Proyecto #2

Documentación

Adrián Mora Rivera – 2024800149
Tamara Robles Camacho – 2024099342

I Semestre
Año lectivo 2026

Curso de Bases de Datos II
Profesor: Kenneth Obando

Jueves 25 de mayo de 2026

Tabla de contenido

Parte I. Arquitectura y diseño (el qué y el cómo está construido)	3
1. Introducción	3
2. Diseño general del sistema	4
2.1. Arquitectura lógica	4
2.2. Arquitectura física	6
2.3. Puertos expuestos al host	9
2.4. Volúmenes persistentes	12
3. Microservicios y sus responsabilidades	16
3.1. API principal: CRUD y entidades	16
3.2. Microservicio de búsqueda: WS Search	19
3.3. Microservicio de grafos y rutas: Rutas-service	20
3.4. Nginx: balanceador de carga y proxy inverso	23
3.5. Keycloak: servidor de autenticación	24
3.6. Redis: caché distribuida	25
3.7. MongoDB: base de datos documental sharded	26
3.8. Elasticsearch: motor de búsqueda	28
3.9. Neo4J: base de datos de grafos	29
3.10. Stack analítico OLAP	30
 Parte II. Funcionalidad (el cómo se usa y qué hace)	 33
4. Flujo de datos y balanceador de carga	33
4.1. Flujo CRUD con Caché Redis	33
4.2. Flujo de búsqueda con Elasticsearch	34
4.3. Flujo de análisis de grafos y rutas	34
4.4. Uso del balanceador de carga	35
5. Análisis de grafos con Neo4J	36
5.1. Modelo del grafo	36
5.2. Ubicaciones geográficas	38
5.3. Consultas Cypher implementadas	38
5.3.1. Co-compras — los 5 pares de platos más comprados juntos	38
5.3.2. Usuarios influyentes	39
5.3.3. Camino mínimo con Dijkstra	39
5.4. Script de migración	40
6. Asignación de rutas de entrega	41
6.1. Flujo de procesamiento	41
6.2. Algoritmo de vecino más cercano	41
7. Pipeline OLAP	43
7.1. Orquestación con Apache Airflow	44
7.2. Data Warehouse con Apache Hive	45
7.2.1. Tablas de hechos	45

7.2.2. Dimensiones	48
7.2.3. Vistas OLAP	48
7.3. Procesamiento con Apache Spark	48
7.4. Visualización con Apache Superset	49
8. Referencia de API	51
8.1. Grupos de endpoints	51
8.1.2. Microservicio de búsqueda — prefijo /search/	52
8.1.3. Microservicio de grafos y rutas — prefijos /grafos/ y /rutas/	52
8.2. Autenticación	52
Parte III. Operación	53
9. Comandos de operación	53
9.1. Docker Compose	53
9.1.1. Stack principal con PostgreSQL es:	53
9.1.2. Stack principal con MongoDB	54
9.1.3. Stack analítico OLAP	55
9.1.4. Stack de Neo4J y rutas-service	56
9.1.5. Cambiar entre motores de base de datos	57
9.1.6. Gestión del DAG de Airflow	58
9.1.7. Generación de datos de prueba	59
9.1.8. Verificación de integridad del Data Warehouse	59
9.1.9. URLs de acceso — Docker Compose	60
9.2. Kubernetes	61
9.2.1. Primer despliegue	61
9.2.2. Despliegue incremental	61
9.2.3. Acceso a servicios en Kubernetes	62
9.2.4. Escalar servicios	62
9.2.5. Cambiar motor de base de datos en Kubernetes	63
9.2.6. Limpiar todo	63
9.3. CI/CD	63
10. Decisiones de diseño	64
10.1. Arquitectura y separación de responsabilidades	64
10.2. Base de datos y persistencia	65
10.3. Rendimiento y escalabilidad	66
10.4. Orquestación y operación	66
11. Tecnologías utilizadas	68
12. Anexos	70
12.1. Anexo A: video demo (tercera etapa)	70
12.2. Anexo B: documentación completa de API	70
12.3. Anexo C: video demo (segunda etapa)	70

Parte I. Arquitectura y diseño (el qué y el cómo está construido)

1. Introducción

El presente documento sirve como la documentación oficial del proyecto 2 – Análisis y Procesamiento OLAP desarrollado en el curso de Bases de Datos II. Este proyecto es continuación directa del Proyecto 1 (Sistema de Reserva Inteligente de Restaurantes), sobre cuya arquitectura de microservicios se construyen nuevas capacidades de análisis de datos.

Los objetivos anteriores del proyecto 1 son incorporar una arquitectura de microservicios con soporte para múltiples motores de base de datos, búsqueda avanzada, caché distribuida y balanceo de carga, además de un pipeline para ejecutar pruebas, construir imágenes Docker y almacenarlas en el Container Registry del repositorio. Esto se logra con las siguientes tecnologías: CI/CD para el pipeline, MongoDB con replicación y sharding, autenticación de usuarios con Keycloak, la base de datos PostgreSQL para almacenar los datos de Keycloak y también como posible base de datos para el proyecto (se puede elegir entre PostgreSQL y MongoDB), Elasticsearch como servidor de búsqueda, Redis como cache, Nginx como balanceador de carga y escalabilidad con Kubernetes.

Añadiendo a lo anterior, los objetivos adicionales de esta continuación son implementar capacidades de análisis OLAP, procesamiento de grandes volúmenes de datos con Apache Spark, visualización de datos, orquestación con Apache Airflow, análisis de grafos con Neo4J y asignación optimizada de rutas de entrega.

Con este escrito, se pretende informar sobre las herramientas utilizadas en el proyecto, desde el diseño general del sistema, su arquitectura lógica y física, hasta los microservicios, sus responsabilidades, el flujo de datos y el uso del balanceador.

Finalmente, cabe aclarar que el diseño sigue principios SOLID, el patrón DAO para la abstracción de la capa de datos y el patrón Strategy para la seleccionar de manera dinámica el motor de base de datos con variables de entorno y sin modificar código fuente.

2. Diseño general del sistema

El sistema está diseñado siguiendo una arquitectura de microservicios, donde cada componente cumple una responsabilidad específica y se comunica a través de la red interna del clúster. Un balanceador de carga centraliza el acceso externo y enruta las solicitudes hacia los servicios correspondientes.

En esta tercera etapa, se incorporan capacidades analíticas OLAP, análisis de grafos con Neo4J y asignación optimizada de rutas de entrega, que se integran como nuevas capas sin modificar la arquitectura operacional existente. Todos los microservicios son independientes y se comunican a través de la red interna del clúster Docker, lo que facilita el mantenimiento, la escalabilidad horizontal y la separación de responsabilidades.

2.1. Arquitectura lógica

En el siguiente diagrama (Figura 1), se muestra la arquitectura lógica del sistema, en donde se especifica qué componentes tiene el sistema y cómo se comunica, sin importar en qué máquinas corren.

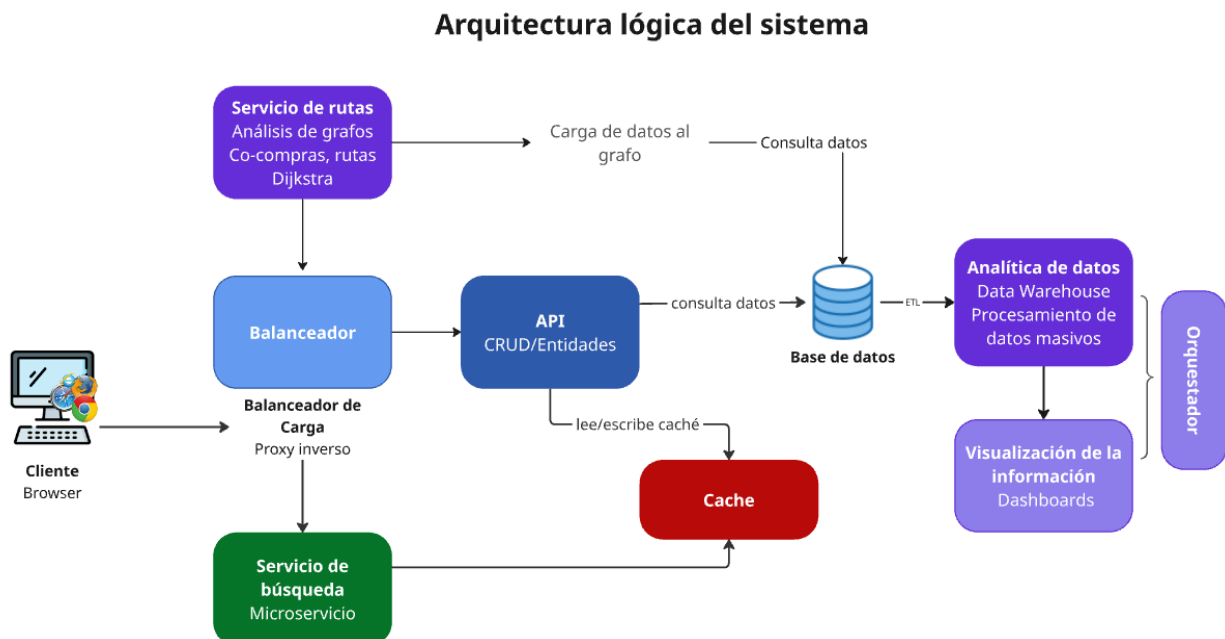


Figura 1. Diagrama de arquitectura lógica

El sistema se compone de los siguientes módulos lógicos:

- **Gestión del dominio:** componente responsable de administrar las entidades principales del negocio: usuarios, restaurantes, menús, pedidos y reservaciones.

Provee las operaciones de creación, consulta, actualización y eliminación, además de aplicar las reglas de negocio asociadas a cada entidad.

- **Búsqueda y descubrimiento:** módulo especializado en la consulta de información mediante criterios de texto libre y categorías. Permite localizar restaurantes, menús y otros recursos de forma eficiente sin afectar el desempeño del módulo transaccional.
- **Control de acceso e identidad:** componente encargado de la autenticación de usuarios y la autorización de operaciones. Gestiona el ciclo de vida de las credenciales, los roles y permisos del sistema, proporcionando mecanismos para verificar la identidad de los usuarios en las operaciones protegidas.
- **Gestión de rendimiento y acceso rápido a datos:** módulo orientado a optimizar el tiempo de respuesta del sistema mediante el almacenamiento temporal de información de alta consulta, reduciendo la carga sobre los componentes transaccionales.
- **Análisis de relaciones y optimización de rutas:** componente encargado de modelar y analizar relaciones entre usuarios, pedidos, productos y ubicaciones geográficas. Permite identificar patrones de interacción, calcular métricas de influencia, descubrir relaciones relevantes y generar rutas de entrega optimizadas.
- **Integración y sincronización de datos analíticos:** módulo responsable de extraer información operativa, transformarla y consolidarla en una estructura orientada al análisis histórico. Su función es desacoplar las operaciones transaccionales de las actividades analíticas.
- **Procesamiento analítico:** componente encargado de ejecutar cálculos agregados y análisis sobre los datos históricos, generando indicadores y métricas que apoyan la toma de decisiones.
- **Visualización y explotación de información:** módulo que presenta los resultados analíticos mediante reportes y tableros interactivos, permitiendo explorar tendencias de consumo, comportamiento de clientes, horarios de mayor demanda y otros indicadores estratégicos.
- **Enrutamiento y exposición de servicios:** componente que centraliza el acceso a los distintos servicios del sistema, dirigiendo las solicitudes hacia el módulo correspondiente y proporcionando un punto único de entrada para los clientes.

2.2. Arquitectura física

La Figura 2 muestra la arquitectura física del sistema en su estado final, incorporando los componentes de la etapa 3. A diferencia de la arquitectura lógica que describe qué componentes existen y cómo se comunican independientemente de dónde corren, la arquitectura física especifica exactamente cómo y dónde está desplegado cada componente: qué contenedor lo implementa, en qué puerto escucha y a qué capa pertenece.

Todos los servicios corren como contenedores Docker orquestados con Docker Compose en una red interna compartida (`proyecto2-basesii_default`). La comunicación entre contenedores usa nombres de servicio como `hostname`, nunca IPs directas. El sistema está también preparado para despliegue en Kubernetes, con manifiestos en `deploy/local/k8s/` para todos los componentes.

Arquitectura física del sistema

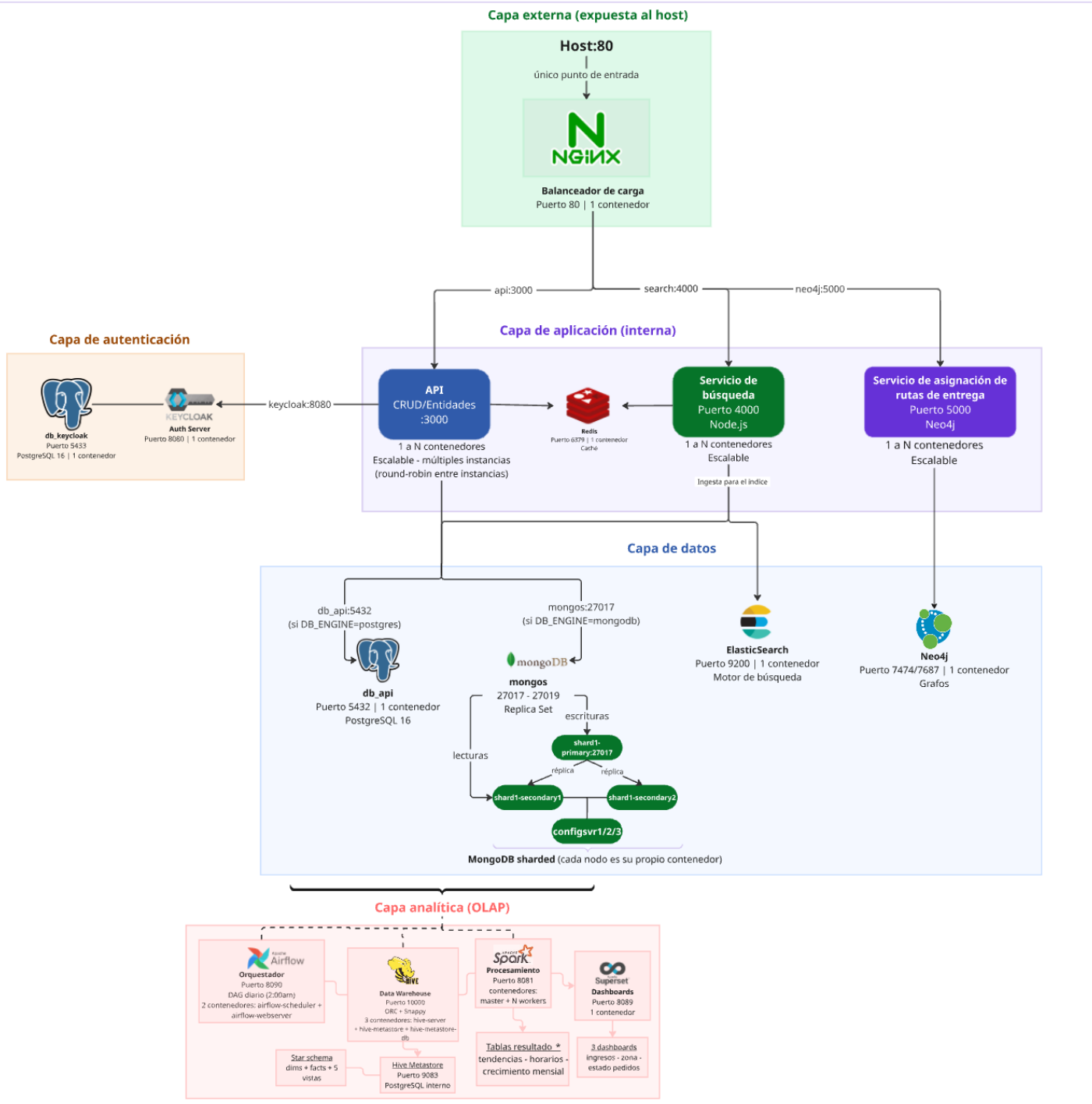


Figura 2. Diagrama de arquitectura física del sistema

El diagrama organiza los componentes en cinco capas:

1. Capa externa.

Nginx actúa como único punto de entrada en el puerto 80, enrutando /api/ hacia la API principal, /search/ hacia el microservicio de búsqueda, y /grafos/ y /rutas/ hacia el microservicio de análisis de grafos.

2. Capa de aplicación.

Tres microservicios independientes: la API CRUD (Node.js, escalable horizontalmente con round-robin), el microservicio de búsqueda (Node.js + Elasticsearch) y el microservicio de rutas-service (Node.js + Neo4J). Todos comparten Redis como caché distribuida.

3. Capa de autenticación

Keycloak gestiona la identidad y emite tokens JWT firmados con RS256. Usa su propia instancia de PostgreSQL aislada de los datos de la aplicación.

4. Capa de datos

El motor de base de datos operacional es intercambiable mediante la variable DB_ENGINE: PostgreSQL 16 (instancia única) o MongoDB 7 en configuración sharded (mongos + 3 config servers + shard con primario y dos secundarios). Elasticsearch indexa el catálogo de productos y Neo4J almacena el grafo de relaciones entre usuarios, pedidos y ubicaciones geográficas de Cartago.

5. Capa analítica OLAP

Componente nuevo de la etapa 3. Airflow orquesta un pipeline ETL diario que extrae datos del motor OLTP activo, los carga en Apache Hive bajo un esquema estrella, y lanza tres jobs de Apache Spark que calculan tendencias de consumo, horarios pico y crecimiento mensual. Los resultados son visualizados por Apache Superset en tres dashboards interactivos. Esta capa es agnóstica del motor OLTP activo: el mismo DAG de Airflow, el mismo esquema estrella en Hive y los mismos dashboards de Superset funcionan tanto con PostgreSQL como con MongoDB, cambiando únicamente la variable DB_ENGINE en el archivo de configuración.

2.3. Puertos expuestos al host

El sistema expone únicamente los servicios necesarios al host para garantizar seguridad y control de acceso. La filosofía de diseño es que Nginx funcione como el único punto de entrada para el tráfico de la aplicación, mientras que los demás servicios permanecen internos al clúster y solo se exponen mediante port-forwarding cuando se necesita acceso directo para administración, depuración o desarrollo.

En la Etapa 3 se incorporaron nuevos servicios analíticos y de grafos que también siguen este principio: Neo4J, Airflow, Hive, Spark y Superset son accesibles directamente desde el host para facilitar el desarrollo y la demostración, pero en un entorno de producción real solo estarían disponibles dentro de la red interna. (Ver Tabla 1).

Contenedor	Puerto interno	Puerto host	Descripción
nginx	80	80	Punto de entrada único, — enruta /api/, /search/, /grafos/, /rutas/
api	3000	Interno	API CRUD (escalable, , acceso solo via Nginx)
search-service	4000	Interno	Microservicio búsqueda (acceso solo via Nginx)
rutas-service	5000	5050	Microservicio de grafos y rutas en Neo4j
keycloak	8080	8080	Consola de administración y endpoint de tokens
db_api	5432	5435	PostgreSQL — base de datos de la aplicación
db_keycloak	5432	5433	PostgreSQL — base de datos exclusiva de Keycloak
mongo1	27017	27017	MongoDB primario
mongo2	27017	27018	MongoDB secundario 1
mongo3	27017	27019	MongoDB secundario 2
redis	6379	6379	Caché Redis
elasticsearch	9200	9200	Motor de búsqueda

Contenedor	Puerto interno	Puerto host	Descripción
neo4j	7474	7474	Neo4J Browser (interfaz web)
neo4j	7687	7687	Neo4J Bolt protocol (driver de aplicación)
airflow-webserver	8080	8090	Interfaz web de Apache Airflow
hive-server	10000	10000	HiveServer2 — endpoint JDBC/Thrift
hive-server	10002	10002	Hive Web UI
spark-master	8080	8081	Spark Master UI
spark-master	7077	7077	Spark cluster endpoint (para spark-submit)
superset	8088	8089	Apache Superset — dashboards analíticos

Tabla 1. Puertos expuestos al host | Docker Compose

Nótese que api y search-service no exponen puertos directamente al host – todo su tráfico pasa obligatoriamente por Nginx, lo que garantiza que el balanceo de carga round-robin se aplique incluso cuando se accede desde herramientas como Postman. El puerto 5435 de PostgreSQL se expone con un número distinto al estándar para evitar conflictos con instalaciones locales de PostgreSQL en la máquina del desarrollador. Lo mismo aplica para los puertos de Keycloak (8090 en Airflow para evitar colisión con el 8080 de Keycloak) y Superset (8089 para evitar colisión con otros servicios web).

En la siguiente tabla (Tabla 2), se muestran los puertos expuestos al host – Kubernetes. En Kubernetes, la política de exposición es más estricta. Solo Nginx y Keycloak tienen servicios con acceso externo. El resto de los componentes son de tipo ClusterIP y requieren kubectl port-forward para acceso desde el host.

Servicio K8s	Tipo	Puerto	Acceso desde el host
nginx-service	LoadBalancer	80	http://localhost (via minikube tunnel)
keycloak-service	ClusterIP	8080	kubectl port-forward svc/keycloak-service 9999:8080
neo4j-service	ClusterIP	7474/7687	kubectl port-forward svc/neo4j-service 7474:7474
airflow-webserver	ClusterIP	8080	kubectl port-forward svc/airflow-webserver 8090:8080
superset	ClusterIP	8088	kubectl port-forward svc/superset 8089:8088
hive-server	ClusterIP	10000/ 10002	kubectl port-forward svc/hive-server 10000:10000
Todos los demás	ClusterIP	-	Internos al cluster, no accesibles desde el host

Tabla 2. Puertos expuestos al host | Kubernetes

2.4. Volúmenes persistentes

Los volúmenes persistentes garantizan que los datos sobrevivan reinicios de contenedores. Sin ellos, cada vez que un contenedor se detiene o se recrea todos sus datos se pierden. El sistema usa volúmenes Docker nombrados — en lugar de bind mounts — porque son gestionados directamente por Docker, son portables entre máquinas y su ciclo de vida es independiente del contenedor que los usa.

Asimismo, el sistema tiene dos categorías de volúmenes:

- a. Los volúmenes operacionales, que persisten los datos de la aplicación en tiempo real: bases de datos, índices de búsqueda, caché y configuración de autenticación.
- b. Los volúmenes analíticos, que persisten el Data Warehouse, los metadatos de Hive, los logs de Airflow y los dashboards de Superset; datos que se reconstruyen vía el pipeline ETL pero cuya pérdida implicaría volver a correr el DAG completo.

A continuación, en la Tabla 3 se resumen los volúmenes persistentes para Docker Compose.

Volumen	Montado en	Servicio	Propósito
pgdata_api	/var/lib/postgresql/data	db_api	Datos de la aplicación en PostgreSQL
pgdata_keycloak	/var/lib/postgresql/data	db_keycloak	Base de datos exclusiva de Keycloak
mongo1_data	/data/db	shard1-primary	Datos del nodo primario del shard rs0
mongo1_secondary1_data	/data/db	shard1-secondary1	Réplica secundaria 1 del shard rs0
mongo1_secondary2_data	/data/db	shard1-secondary2	Réplica secundaria 2 del shard rs0
configsvr1_data	/data/db	configsvr1	Config server 1 del cluster sharded
configsvr2_data	/data/db	configsvr2	Config server 2 del cluster sharded
configsvr3_data	/data/db	configsvr3	Config server 3 del cluster sharded

Volumen	Montado en	Servicio	Propósito
redis_data	/data	redis	Persistencia de caché Redis (AOF)
es_data	/usr/share/elasticsearch/data	elasticsearch	Índices de Elasticsearch
neo4j_data	/data	neo4j	Grafo de Neo4J (nodos, relaciones, propiedades)
neo4j_logs	/logs	neo4j	Logs de Neo4J
neo4j_plugins	/plugins	neo4j	Plugins APOC y Graph Data Science (GDS)
hive_warehouse_data	/opt/hive/data/warehouse	hive-server, spark-master, spark-worker	Archivos ORC del Data Warehouse (tablas y particiones)
hive_metastore_data	/var/lib/postgresql/data	hive-metastore-db	PostgreSQL interno del metastore de Hive
airflow_db_data	/var/lib/postgresql/data	airflow-db	Base de datos de estado de Airflow (DAGs, runs, tasks)
airflow_logs	/opt/airflow/logs	airflow-scheduler, airflow-webserver	Logs de ejecución de cada tarea del DAG
superset_data	/app/superset_home	superset	Metadata de Superset: dashboards, datasets, conexiones

Tabla 3. Volúmenes persistentes | Docker Compose

Por su parte, cabe destacar tres volúmenes con comportamiento especial:

- El volumen `hive_warehouse_data` es compartido entre `hive-server`, `spark-master` y `spark-worker` — los tres necesitan acceso al mismo sistema de archivos porque Spark lee y escribe directamente los archivos ORC del warehouse sin pasar por `HiveServer2`.
- El volumen `neo4j_plugins` persiste los plugins APOC y GDS que Neo4J descarga al primer arranque; sin este volumen los plugins se descargarían en cada reinicio, lo que tarda varios minutos.

- c. El volumen airflow_logs es crítico para la trazabilidad del pipeline: contiene el log detallado de cada tarea de cada run del DAG, incluyendo los outputs de los jobs de Spark.

En la siguiente tabla (Tabla 4) se muestran los datos que persisten en volúmenes persistentes nombrados en Kubernetes para sobrevivir reinicios:

PVC	Montado en	Servicio	Tamaño
postgres-api-pvc	/var/lib/postgresql/data	PostgreSQL API	5 Gi
keycloak-postgres-data	/var/lib/postgresql/data	PostgreSQL Keycloak	2 Gi
configsvr-data-configsvr-0	/data/configdb	MongoDB config server	1 Gi
shard-data-shard-0	/data/db	MongoDB shard primario	10 Gi
es-data-elasticsearch-0	/usr/share/elasticsearch/data	ElasticSearch	5 Gi
redis-pvc	/data	Redis	1 Gi
neo4j-data-pvc	/data	Neo4J	5 Gi
neo4j-logs-pvc	/logs	Neo4J	1 Gi
neo4j-plugins-pvc	/plugins	Neo4J	1 Gi
hive-warehouse-pvc	/opt/hive/data/warehouse	Hive + Spark	20 Gi
hive-metastore-pvc	/var/lib/postgresql/data	Hive Metastore DB	2 Gi
airflow-db-pvc	/var/lib/postgresql/data	Airflow DB	2 Gi
airflow-logs-pvc	/opt/airflow/logs	Airflow Scheduler + Webserver	5 Gi
superset-data-pvc	/app/superset_home	Superset	2 Gi

Tabla 4. Volúmenes persistentes | Kubernetes (PVCs)

Por añadidura, en Kubernetes los volúmenes se gestionan como PersistentVolumeClaims (PVCs). A diferencia de Docker donde los volúmenes se crean automáticamente, los PVCs en Kubernetes deben declararse explícitamente con el tamaño de almacenamiento requerido y el modo de acceso.

El PVC de mayor tamaño es hive-warehouse-pvc con 20 Gi, dado que almacena todos los archivos ORC del Data Warehouse incluyendo las tablas de hechos particionadas por año y mes, las dimensiones y las tablas de resultados generadas por Spark. En un entorno de producción real con mayor volumen de datos este tamaño debería ajustarse según la tasa de crecimiento estimada.

3. Microservicios y sus responsabilidades

El sistema está construido bajo una arquitectura de microservicios donde cada servicio es independiente, tiene una responsabilidad específica y puede escalar de forma aislada sin afectar al resto. Todos los servicios se comunican a través de la red interna del clúster usando nombres de servicio como hostname, nunca IPs directas. El acceso externo pasa siempre por Nginx, que actúa como reverse proxy y balanceador de carga.

A continuación, se describe cada microservicio con sus características técnicas y responsabilidades.

3.1. API principal: CRUD y entidades

La API principal es el núcleo del sistema. Gestiona todas las entidades del dominio y es el único servicio que se comunica directamente con la base de datos operacional. La Tabla 5 resume sus características técnicas.

Atributo	Valor
Tecnología	Node.js 20, Express 5
Puerto interno	3000
Imagen Docker	ghcr.io/adrian-mora-05/tareacorta1-basesii:latest
Escalabilidad	Horizontal --scale api=N con Nginx como balanceador round-robin
Base de datos	PostgreSQL 16 o MongoDB 7, seleccionable via DB_ENGINE
Autenticación	JWT validado contra Keycloak en cada petición protegida

Tabla 5. Características técnicas de la API principal

Responsabilidades:

- Gestión completa (CRUD) de restaurantes, menús, usuarios, reservaciones y pedidos.
- Autenticación y registro de usuarios a través de Keycloak usando OAuth2/OIDC.
- Validación de tokens JWT en cada petición protegida, consultando el endpoint JWKS de Keycloak.
- Caché de respuestas frecuentes en Redis con políticas TTL e invalidación por evento (patrón Cache-Aside), según las políticas descritas en la Tabla 6.
- Abstracción del motor de base de datos mediante el patrón DAO — el único punto del sistema donde se evalúa DB_ENGINE es DAOFactory.js.

Recurso	TTL	Invalidación
Restaurantes	300 segundos (5 minutos)	Al crear o modificar un restaurante
Menús	180 segundos (3 minutos)	Al crear, modificar o eliminar un menú
Resultados de búsqueda	60 segundos (1 minuto)	Al reindexar productos

Tabla 6. Políticas de caché Redis en la API principal

La estructura interna de la API sigue el patrón Controller → Service → DAOFactory → DAO, donde los controladores reciben el servicio por inyección de dependencias y los servicios reciben el DAO ya instanciado sin saber si viene de PostgreSQL o MongoDB. Esto cumple el principio DIP de SOLID y facilita las pruebas unitarias con mocks.

Adicionalmente, refiérase a la Figura 4 para el modelo relacional de datos de la base de datos con las entidades y catálogos que la conforman.

Diagrama de BD-Postgres

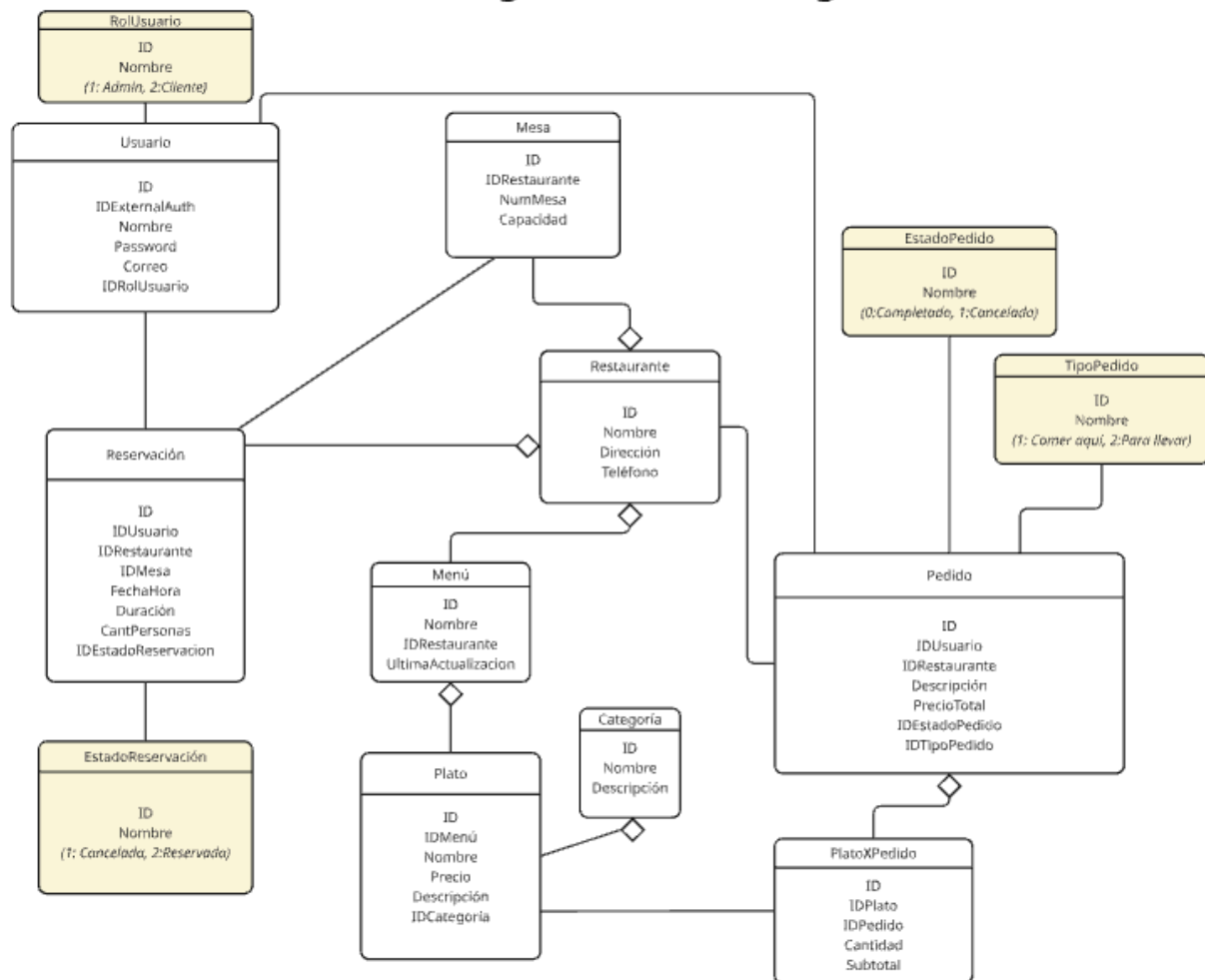


Figura 5. Modelo de datos en la base de datos Postgres

3.2 Microservicio de búsqueda: WS Search

El microservicio de búsqueda es independiente de la API principal y tiene como única responsabilidad la búsqueda textual sobre el catálogo de productos. La Tabla 7 resume sus características.

Atributo	Valor
Tecnología	Node.js 20, Express, @elastic/elasticsearch
Puerto interno	4000
Motor de índice	ElasticSearch 8.13 — índice: products
Dependencias	Solo ElasticSearch (no depende de la BD principal)

Tabla 7. Características técnicas del microservicio de búsqueda

Responsabilidades:

- Exponer endpoints de búsqueda textual y por categoría sobre el índice products.
- Reindexar el catálogo de productos desde la base de datos de la API cuando se solicite.
- Aplicar el valor por defecto "Producto sin descripción" cuando un plato no tiene descripción.
- Mantener la separación de responsabilidades — la búsqueda no es responsabilidad de la API CRUD.

Las búsquedas usan una query `multi_match` sobre los campos nombre, categoría y descripción. Una coincidencia en el nombre tiene mayor peso que en la categoría, y esta mayor peso que en la descripción. Los resultados vienen ordenados por relevancia mediante el campo `score`.

3.3 Microservicio de grafos y rutas: Rutas-service

El microservicio de rutas y grafos cubre los puntos 5 y 6 del enunciado: análisis de grafos con Neo4J y asignación optimizada de rutas de entrega. La Tabla 8 resume sus características técnicas.

Atributo	Valor
Tecnología	Node.js 20, Express, neo4j-driver 5.x, pg 8.x, mongodb 6.x
Puerto interno	5000
Puerto host (Docker)	5050
Acceso via Nginx	/grafos/** y /rutas/**
Dependencias	Neo4J, PostgreSQL o MongoDB (según DB_ENGINE)

Tabla 8. Características técnicas de rutas-service

Responsabilidades:

- Exponer endpoints REST para consultar el grafo de Neo4J (co-compras, usuarios influyentes, caminos mínimos).
- Leer pedidos pendientes del motor de base de datos activo (PostgreSQL o MongoDB según DB_ENGINE).
- Simular geolocalización de clientes asignando ubicaciones de las 8 zonas de Cartago.
- Asignar pedidos a repartidores con distribución round-robin para carga equitativa.
- Calcular el orden óptimo de visita por repartidor con el algoritmo de vecino más cercano.

La Tabla 9 describe el modelo del grafo en Neo4J: los tipos de nodos y sus propiedades.

Nodo	Propiedades	Descripción
Usuario	id, nombre, correo, rol	Usuarios del sistema
Pedido	id, descripcion, id_restaurante	Pedidos realizados
Plato	id, nombre, categoria, precio, descripcion	Productos del menú
Restaurante	id, nombre, direccion	Restaurantes registrados
Ubicacion	id, nombre, lat, lng	Zonas geográficas de Cartago con coordenadas reales

Tabla 9. Nodos del grafo Neo4J

La Tabla 10 describe las relaciones entre nodos, incluyendo sus propiedades cuando las tienen.

Relación	De → A	Propiedades	Descripción
HIZO	Usuario → Pedido	—	El usuario realizó este pedido
CONTIENE	Pedido → Plato	cantidad	El pedido incluye este plato
ES_DE	Pedido → Restaurante	—	El pedido pertenece a este restaurante
RECOMIENDA	Usuario → Usuario	—	El usuario recomienda a otro (simulada)
CONECTA	Ubicacion → Ubicacion	distancia_km, tiempo_min	Conexión entre zonas con peso

Tabla 10. Relaciones del grafo Neo4J

A diferencia de la API principal, rutas-service no implementa el patrón DAO completo. Dado que solo necesita una operación de lectura simple sobre pedidos pendientes, una conexión directa con pg y mongodb es suficiente y evita sobreingeniería en un microservicio de análisis.

Las 8 ubicaciones del grafo corresponden a zonas reales de la provincia de Cartago, cargadas en Neo4J mediante el script neo4j/seed/ubicaciones.cypher. Sus coordenadas y las 11 conexiones bidireccionales entre ellas se describen en la Tabla 11.

Ubicación	ID	Latitud	Longitud
Cartago Centro	cartago-centro	9.8647	-83.9192
Tres Ríos	tres-rios	9.9003	-83.9917
La Unión	la-union	9.9086	-83.9614
San Diego	san-diego	9.8891	-83.9456
Paraíso	paraiso	9.8358	-83.8650
El Tejar	el-tejar	9.8514	-83.9758
San Rafael	san-rafael	9.9200	-83.9300
Agua Caliente	agua-caliente	9.8750	-83.9050

Tabla 11. Ubicaciones y conexiones del grafo de Cartago

3.4 Nginx: balanceador de carga y proxy inverso

La Tabla 12 resume las características de Nginx. Su función principal es actuar como único punto de entrada del sistema, encapsulando la topología interna de contenedores.

Atributo	Valor
Imagen	nginx:alpine
Puerto expuesto	80
Algoritmo de balanceo	Round-robin entre instancias de la API

Tabla 12. Características técnicas de Nginx

Responsabilidades:

- Recibir todas las peticiones externas en el puerto 80 como único punto de entrada.
- Enrutar peticiones al microservicio correcto según el prefijo de la URL: /api/ hacia la API principal, /search/ hacia el microservicio de búsqueda, /grafos/ y /rutas/ hacia rutas-service.
- Distribuir la carga entre las N instancias de la API con round-robin cuando se escala horizontalmente.
- Ocultar la topología interna de contenedores al cliente externo.

3.5 Keycloak: servidor de autenticación

Keycloak gestiona toda la identidad del sistema. La Tabla 13 resume sus características técnicas.

Atributo	Valor
Imagen	quay.io/keycloak/keycloak:24.0
Puerto	8080
Realm	restaurant
Roles	admin, cliente
Protocolo	OpenID Connect / OAuth2 — tokens JWT firmados con RS256
Base de datos	PostgreSQL dedicado (db_keycloak)

Tabla 13. Características técnicas de Keycloak

Responsabilidades:

- Emitir tokens JWT firmados con RS256 para usuarios autenticados.
- Gestionar usuarios, roles y credenciales del realm restaurant.
- Importar la configuración inicial desde realm-export.json al arrancar.
- Proveer el endpoint JWKS que la API consulta para validar tokens sin estado.

La API nunca almacena contraseñas ni autentica directamente — únicamente valida los JWT emitidos por Keycloak en cada petición protegida.

3.6 Redis: caché distribuida

La Tabla 14 resume las características de Redis. Al ser compartido por todas las instancias de la API, garantiza consistencia del caché incluso cuando la API está escalada horizontalmente.

Atributo		Valor
Imagen		redis:7-alpine
Puerto		6379
Política de eviction	de	allkeys-lru (elimina las claves menos usadas al llegar al límite)
Límite de memoria	de	256 MB
Persistencia		Volumen redis_data montado en /data

Tabla 14. Características técnicas de Redis

3.7 MongoDB: base de datos documental sharded

La Tabla 15 resume las características del cluster de MongoDB. Esta configuración garantiza alta disponibilidad mediante replicación automática y distribución de datos entre shards.

Atributo	Valor
Imagen	mongo:7
Topología	1 mongos + 3 config servers + 1 shard (rs0) con 1 primario y 2 secundarios
Shard key	id_restaurante (hashed) en colecciones pedidos y reservaciones
Colecciones	pedidos, reservaciones, menus

Tabla 15. Características técnicas del cluster MongoDB

El contenedor mongo-init configura automáticamente el replica set y el sharding al primer arranque usando MERGE para garantizar idempotencia — puede correrse múltiples veces sin duplicar datos.

Refiérase a la Figura 5 para el diagrama de modelo de datos para la base de datos con el motor de MongoDB.

Diagrama de BD-Mongo

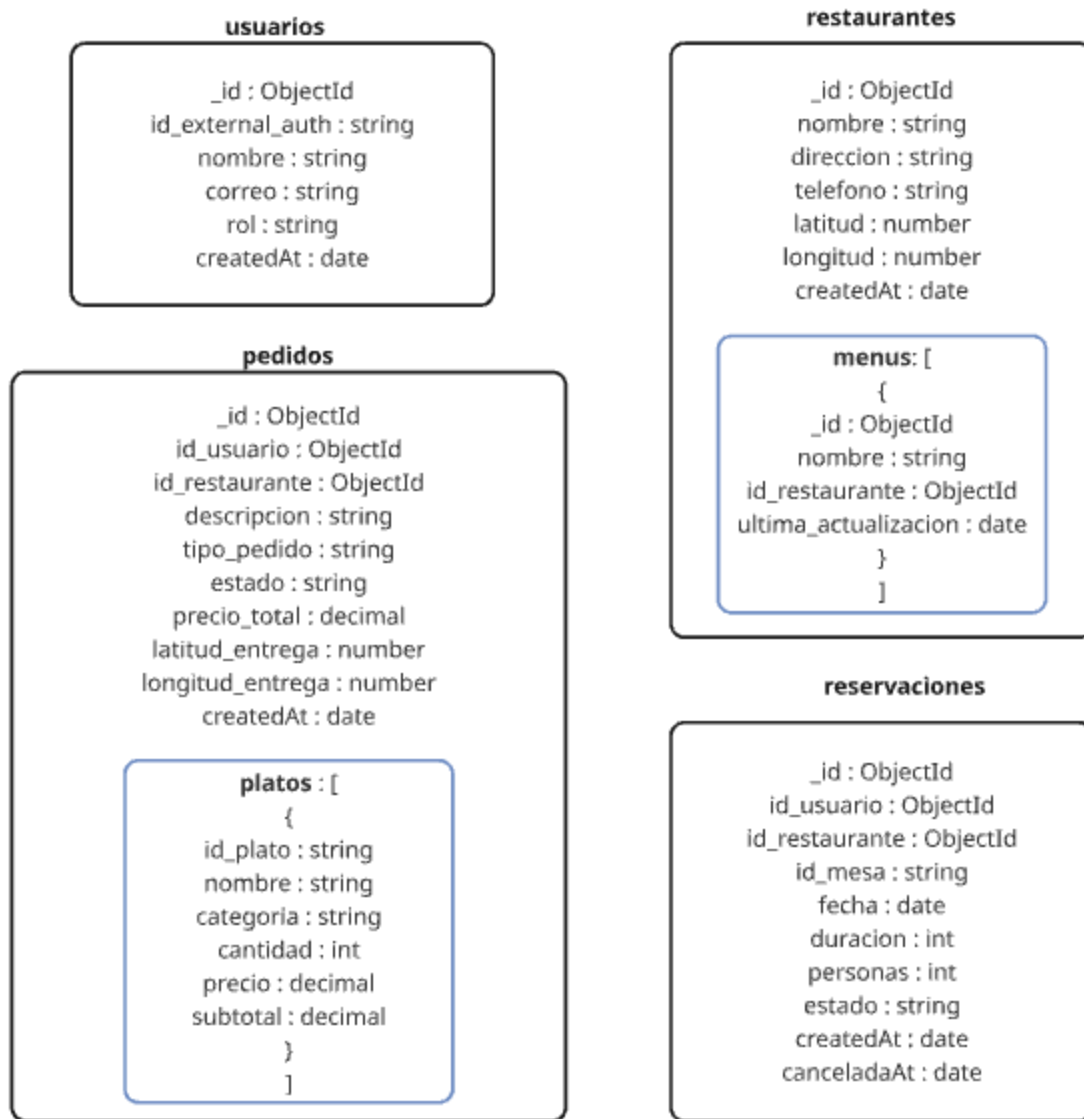


Figura 5. Modelo de datos en la base de datos Mongo

3.8 Elasticsearch: motor de búsqueda

La Tabla 16 resume las características de Elasticsearch. Funciona en modo single-node con seguridad deshabilitada, configuración adecuada para el entorno académico del proyecto.

Atributo	Valor
Imagen	docker.elastic.co/elasticsearch/elasticsearch:8.13.0
Puerto	9200
Índice	products
Configuración	Single-node, security disabled

Tabla 16. Características técnicas de Elasticsearch

3.9 Neo4J: base de datos de grafos

La Tabla 17 resume las características de Neo4J. Los plugins APOC y GDS son fundamentales para las funcionalidades de análisis de grafos y cálculo de caminos mínimos con Dijkstra.

Atributo	Valor
Imagen	neo4j:5.18
Puerto HTTP	7474 (Neo4J Browser)
Puerto Bolt	7687 (driver de aplicación)
Plugins	APOC (utilidades), Graph Data Science — GDS (algoritmos de grafos)

Tabla 17. Características técnicas de Neo4J

El script neo4j/migrate.js lee datos del motor activo según DB_ENGINE y los carga en Neo4J usando MERGE para garantizar idempotencia.

3.10 Stack analítico OLAP

El pipeline OLAP es el conjunto de componentes nuevos incorporados en la Etapa 3. Está compuesto por cuatro servicios que trabajan en secuencia y son completamente agnósticos del motor OLTP activo. La Tabla 18 resume sus características técnicas.

Componente	Contenedores	Versión	Puerto host	Rol
Apache Hive	hive-server, hive-metastore, hive-metastore-db	4.0.0	10000, 10002	Data Warehouse
Apache Spark	spark-master, spark-worker	3.5.6	8081, 7077	Procesamiento batch
Apache Airflow	airflow-scheduler, airflow-webserver, airflow-db	2.9.1	8090	Orquestación ETL
Apache Superset	superset	3.1.0	8089	Visualización

Tabla 18. Componentes del stack analítico OLAP

Apache Hive implementa el Data Warehouse con esquema estrella. Las tablas se almacenan en formato ORC con compresión Snappy, particionadas por año y mes. El metastore usa PostgreSQL interno (hive-metastore-db) para persistir los metadatos del schema. HiveServer2 expone el endpoint JDBC/Thrift en el puerto 10000, que usan tanto Superset como el DAG de Airflow para ejecutar queries.

Apache Spark ejecuta los tres jobs de análisis sobre el Data Warehouse. Se conecta directamente al sistema de archivos del warehouse via el volumen compartido hive_warehouse_data y al metastore via Thrift en el puerto 9083. Los jobs son invocados por Airflow mediante docker exec spark-master spark-submit, por lo que el scheduler de Airflow necesita acceso al socket Docker (/var/run/docker.sock).

Apache Airflow orquesta el pipeline ETL con ejecución diaria a las 02:00 AM. El DAG restaurant_pipeline implementa diez tareas en secuencia: detección del motor activo mediante un BranchPythonOperator, extracción desde el OLTP, normalización del schema, carga de las seis dimensiones, carga de las dos tablas de hechos, tres jobs de

Spark en paralelo y reindexación de ElasticSearch al finalizar. La Tabla 19 describe las tareas del DAG en orden de ejecución.

Tarea	Tipo	Descripción
branch_engine	BranchPythonOperator	Lee DB_ENGINE y decide la rama de extracción
extract_postgres	PythonOperator	Extrae datos desde PostgreSQL
extract_mongo	PythonOperator	Extrae datos desde MongoDB (rama alternativa)
join	PythonOperator	Converge las dos ramas
unify	PythonOperator	Normaliza el schema de salida
load_dims	PythonOperator	Carga las 6 dimensiones en Hive (INSERT OVERWRITE)
load_facts	PythonOperator	TRUNCATE + INSERT INTO en fact_pedido y fact_reservacion
spark_tendencias_consumo	PythonOperator	Job Spark: ingresos por mes y categoría, variación MoM
spark_horarios_pico	PythonOperator	Job Spark: demanda por hora y día de semana
spark_crecimiento_mensual	PythonOperator	Job Spark: MoM, YTD y tasa de cancelación
reindex_elasticsearch	PythonOperator	Reindexación del catálogo de productos

Tabla 19. Tareas del DAG restaurant_pipeline

Apache Superset visualiza los resultados del pipeline mediante tres dashboards que consumen las tablas resultado_* generadas por Spark. La Tabla 20 describe los dashboards y sus charts.

Dashboard	Charts
Ingresos por mes y categoría	Bar chart apilado por mes y categoría, pie chart de distribución total, big number de ingreso acumulado
Actividad de clientes por zona geográfica	Bar chart por zona cliente vs restaurante, big number de clientes únicos, tabla de desglose
Pedidos completados vs cancelados	Bar chart apilado por mes, pie chart de proporción global, big number de tasa de cancelación

Tabla 20. Dashboards de Apache Superset

Los dashboards están exportados como archivos YAML en `analytics/superset/dashboards/` e importables directamente en cualquier instancia de Superset 3.x.

Parte II. Funcionalidad (el cómo se usa y qué hace)

4. Flujo de datos y balanceador de carga

En esta sección se describe el flujo general de las peticiones dentro del sistema y el rol de Nginx como balanceador de carga y punto de entrada único. Todas las solicitudes HTTP son recibidas por Nginx, que las enruta al servicio correspondiente según el prefijo de la URL: la API principal, el microservicio de búsqueda o el microservicio de grafos y rutas. Para mejorar el rendimiento, la API utiliza Redis como caché distribuida, evitando consultas repetitivas a la base de datos, mientras que el balanceador distribuye la carga entre múltiples instancias cuando la API se escala horizontalmente.

4.1. Flujo CRUD con Caché Redis

Para optimizar el rendimiento y reducir la carga sobre las bases de datos, la API incorpora una capa de caché distribuida con Redis, aplicando el patrón Cache-Aside. De esta manera, las peticiones de lectura frecuentes pueden resolverse directamente desde memoria cuando existe un cache hit, o bien consultar la base de datos y actualizar el caché cuando ocurre un cache miss.

Cuando un cliente realiza una petición de lectura, por ejemplo GET /api/restaurants:

1. **Cliente** → **Nginx :80**. El cliente envía la petición HTTP al único punto de entrada.
2. **Nginx** → **API :3000**. Nginx enruta la petición al upstream api_cluster usando round-robin entre instancias.
3. **API — checkJwt**. El middleware verifica el token JWT consultando el JWKS de Keycloak.
4. **API — Redis**. El Service busca la clave restaurants:all en Redis.
 - a. **Cache HIT**. Si existe, devuelve los datos inmediatamente sin consultar la BD.
 - b. **Cache MISS**. Si no existe, consulta la BD (PostgreSQL o MongoDB según DB_ENGINE).

5. **BD → API → Redis.** El resultado se guarda en Redis con TTL=300s para futuras peticiones.
6. **API → Cliente.** Se devuelve la respuesta JSON al cliente.

4.2. Flujo de búsqueda con Elasticsearch

Las peticiones con prefijo `/search/` son recibidas por Nginx y redirigidas al microservicio de búsqueda, el cual se encarga de consultar Elasticsearch para ejecutar búsquedas eficientes y textuales.

Cuando un cliente realiza una búsqueda, por ejemplo `GET /search/products?q=pollo`:

1. **Cliente → Nginx :80.** El cliente envía la petición con prefijo `/search/`.
2. **Nginx → Search Service :4000.** Nginx detecta el prefijo `/search/` y enruta al contenedor `search-service`.
3. **Search Service — ES Query.** El controlador construye una query multi-match sobre los campos nombre, categoría y descripción.
4. **ElasticSearch :9200.** Elasticsearch ejecuta la búsqueda en el índice `products` y devuelve los documentos coincidentes ordenados por relevancia.
5. **Search Service → Cliente.** El servicio formatea los resultados y los devuelve como JSON al cliente.

4.3. Flujo de análisis de grafos y rutas

Las peticiones con prefijos `/grafos/` y `/rutas/` son recibidas por Nginx y redirigidas a `rutas-service`. Este microservicio consulta Neo4J para ejecutar análisis de grafos y, en el caso de la asignación de rutas, también lee pedidos pendientes de la base de datos operacional activa.

Cuando un cliente solicita una asignación de rutas con `POST /rutas/asignar`:

1. **Cliente → Nginx :80.** El cliente envía la petición con prefijo `/rutas/`.
2. **Nginx → rutas-service :5000.** Nginx detecta el prefijo y enruta al contenedor `rutas-service`.

3. **Rutas-service** → **BD**. El servicio lee los pedidos pendientes de PostgreSQL o MongoDB según DB_ENGINE.
4. **Rutas-service** → **Neo4J :7687**. Consulta las distancias entre zonas usando el grafo de ubicaciones y el algoritmo Dijkstra del plugin GDS.
5. **Rutas-service**. Aplica el algoritmo de vecino más cercano para calcular el orden óptimo de entrega por repartidor.
6. **Rutas-service** → **Cliente**. Devuelve las asignaciones con rutas optimizadas y distancias totales como JSON.

4.4. Uso del balanceador de carga

Nginx cumple dos funciones principales en el sistema: reverse proxy y balanceador de carga. Todas las peticiones externas llegan primero a Nginx, que decide a qué servicio interno debe enviarlas según el prefijo de la URL. Las solicitudes con /api/ se redirigen a la API principal, las solicitudes con /search/ se envían al microservicio de búsqueda, y las solicitudes con /grafos/ y /rutas/ se redirigen al microservicio rutas-service.

Cuando la API se escala horizontalmente con el comando `docker compose --scale api=N`, Docker Compose crea múltiples instancias del contenedor de la API. Nginx distribuye las peticiones entre estas instancias usando el algoritmo round-robin, lo que evita que una sola instancia reciba toda la carga.

Todas las instancias de la API comparten el mismo Redis, que funciona como caché distribuida. Esto hace que los datos almacenados en caché sean consistentes y accesibles para todas las instancias, sin importar cuál atienda la petición del cliente.

5. Análisis de grafos con Neo4J

Neo4J es la base de datos de grafos del sistema. Almacena las relaciones entre entidades del dominio como nodos y aristas, permitiendo consultas que serían complejas o lentas en bases de datos relacionales tradicionales. El modelo del grafo refleja el dominio de negocio del sistema: usuarios que hacen pedidos, pedidos que contienen platos, y ubicaciones geográficas reales de Cartago conectadas por distancias y tiempos estimados de desplazamiento.

5.1. Modelo del grafo

El grafo está compuesto por cinco tipos de nodos, descritos en la Tabla 21, y cinco tipos de relaciones, descritas en la Tabla 22.

Nodo	Propiedades	Descripción
Usuario	id, nombre, correo, rol	Usuarios del sistema
Pedido	id, descripcion, id_restaurante	Pedidos realizados
Plato	id, nombre, categoria, precio, descripcion	Productos del menú
Restaurante	id, nombre, direccion	Restaurantes registrados
Ubicacion	id, nombre, lat, lng	Zonas geográficas de Cartago con coordenadas reales

Relación	De → A	Propiedades	Descripción
HIZO	Usuario → Pedido	—	El usuario realizó este pedido
CONTIENE	Pedido → Plato	cantidad	El pedido incluye este plato
ES_DE	Pedido → Restaurante	—	El pedido pertenece a este restaurante
RECOMIENDA A	Usuario → Usuario	—	El usuario recomienda a otro (simulada)
CONECTA	Ubicacion → Ubicacion	distancia_km, tiempo_min	Conexión entre zonas con peso

Tabla 22. Relaciones del grafo Neo4J

Asimismo, la Figura 3 muestra la estructura visual del grafo con sus nodos y relaciones.

Diagrama de Neo4j

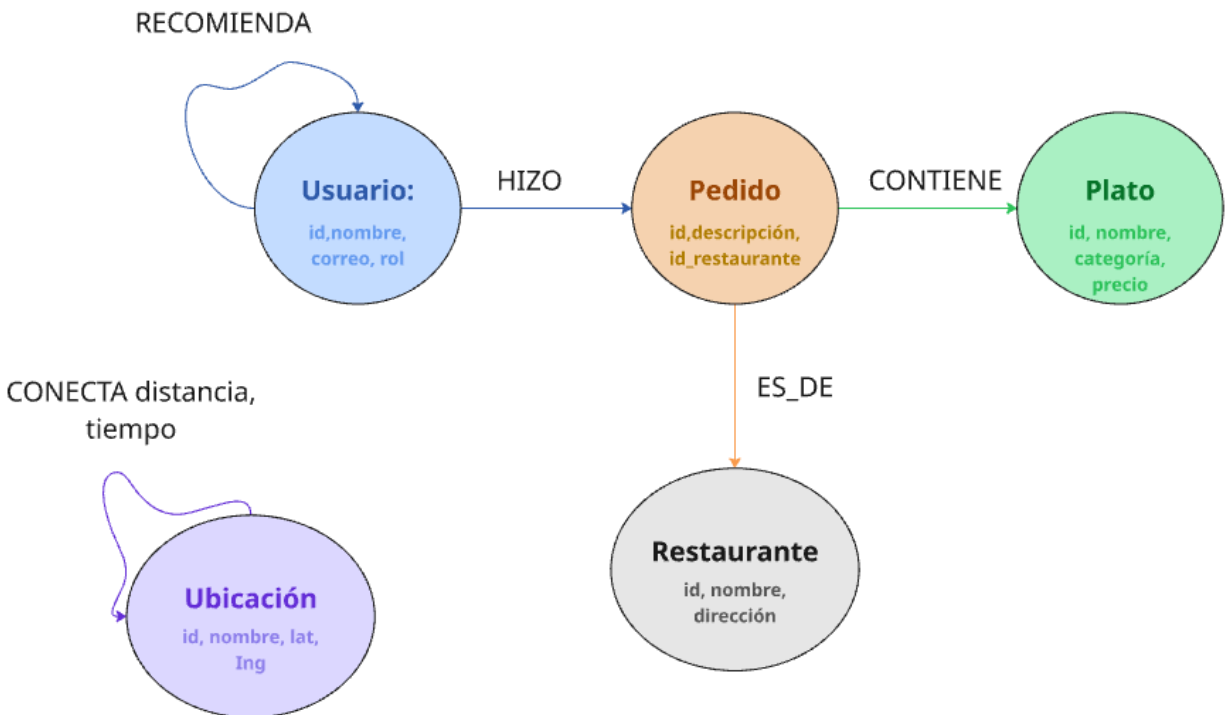


Figura 3. Diagrama de grafos en la base de datos de Neo4j

5.2. Ubicaciones geográficas

El archivo `neo4j/seed/ubicaciones.cypher` carga 8 nodos `Ubicacion` con coordenadas reales de la provincia de Cartago, conectados por 11 relaciones `CONECTA` bidireccionales con distancia en km y tiempo estimado en minutos. Estas ubicaciones son la base del módulo de asignación de rutas de entrega descrito en el capítulo siguiente. La Tabla 23 lista las zonas cargadas.

Ubicación	Latitud	Longitud
Cartago Centro	9.8647	-83.9192
Tres Rios	9.9003	-83.9917
La Union	9.9086	-83.9614
San Diego	9.8891	-83.9456
Paraiso	9.8358	-83.8650
El Tejar	9.8514	-83.9758
San Rafael	9.9200	-83.9300
Agua Caliente	9.8750	-83.9050

Tabla 23. Datos de las ubicaciones cargadas en el grafo

5.3. Consultas Cypher implementadas

El sistema implementa tres consultas Cypher que se exponen como endpoints REST via `routes-service`. A continuación se muestra cada una con su propósito.

5.3.1. Co-compras — los 5 pares de platos más comprados juntos

Identifica qué pares de platos aparecen con mayor frecuencia en el mismo pedido. Útil para implementar recomendaciones del tipo "los que pidieron X también pidieron Y".

```
MATCH
(p1:Plato) <-[:CONTIENE]-(ped:Pedido)-[:CONTIENE]->(p2:Plato)

WHERE id(p1) < id(p2)

RETURN p1.nombre, p2.nombre, COUNT(ped) AS vecesJuntos
```

```
ORDER BY vecesJuntos DESC
```

```
LIMIT 5
```

5.3.2. Usuarios influyentes

Encuentra los usuarios que más recomiendan a otros usuarios, ordenados de mayor a menor cantidad de recomendaciones. Las relaciones RECOMIENDA se crean durante la migración de datos.

```
MATCH (u1:Usuario)-[:RECOMIENDA]->(u2:Usuario)

RETURN u1.nombre,

        COUNT(u2)           AS totalRecomendados,

        COLLECT(u2.nombre) AS recomiendaA

ORDER BY totalRecomendados DESC
```

5.3.3. Camino mínimo con Dijkstra

Calcula el camino más corto entre dos ubicaciones usando el algoritmo Dijkstra del plugin Graph Data Science. Requiere que el grafo rutasGraph esté proyectado en memoria previamente. Se puede optimizar por distancia en kilómetros o por tiempo en minutos cambiando relationshipWeightProperty.

```
MATCH (inicio:Ubicacion { id: $origen })

MATCH (fin:Ubicacion      { id: $destino })

CALL gds.shortestPath.dijkstra.stream('rutasGraph', {

    sourceNode:           inicio,

    targetNode:           fin,

    relationshipWeightProperty: 'distancia_km'

})

YIELD nodeIds, costs
```



```
RETURN [id IN nodeIds | gds.util.asNode(id).id] AS paradas,  
       costs[-1] AS kmTotales
```

5.4. Script de migración

El archivo `neo4j/migrate.js` lee datos del motor de base de datos activo — PostgreSQL o MongoDB según `DB_ENGINE` — y los carga en Neo4J usando `MERGE` para garantizar idempotencia: el script puede ejecutarse múltiples veces sin duplicar datos. Se ejecuta manualmente con:

```
docker compose \  
-f deploy/local/docker/docker-compose.neo4j.yml \  
--project-directory . \  
run neo4j-migrate
```

En el despliegue de Kubernetes, la migración se ejecuta mediante el Job `neo4j-migrate` definido en `deploy/local/k8s/neo4j/migrate-job.yaml`, que corre automáticamente después de que Neo4J esté disponible y el seed de ubicaciones haya completado.

6. Asignación de rutas de entrega

El módulo de asignación de rutas implementa la lógica para distribuir pedidos pendientes entre repartidores disponibles y calcular el orden óptimo de visita para cada uno. Está implementado dentro de rutas-service y se apoya en el grafo de ubicaciones de Neo4J para consultar distancias reales entre zonas de Cartago.

6.1. Flujo de procesamiento

El proceso de asignación sigue los siguientes pasos en orden:

1. Leer los pedidos pendientes de la base de datos activa (PostgreSQL o MongoDB según DB_ENGINE).
2. Simular la geolocalización de cada pedido asignando una ubicación aleatoria de las 8 zonas de Cartago que existen como nodos Ubicacion en Neo4J. Esta simulación reemplaza la geocodificación real de direcciones de clientes, que el sistema no implementa.
3. Distribuir los pedidos entre los repartidores disponibles con algoritmo round-robin, garantizando una carga equitativa independientemente de la cantidad de pedidos.
4. Para cada repartidor, aplicar el algoritmo de vecino más cercano para determinar el orden óptimo de visita sobre los pedidos que le fueron asignados.
5. Para calcular la distancia entre dos puntos, consultar primero si existe una relación CONECTA directa en Neo4J. Si no existe conexión directa, usar Dijkstra sobre el grafo proyectado rutasGraph para encontrar el camino más corto pasando por nodos intermedios.
6. Devolver la ruta optimizada y la distancia total recorrida por repartidor como respuesta JSON.

6.2. Algoritmo de vecino más cercano

La heurística de vecino más cercano es el algoritmo de ruteo implementado para calcular el orden de visita de cada repartidor. Se eligió por su simplicidad de implementación y su buen desempeño en problemas de ruteo con pocos puntos, como es el caso del sistema donde cada repartidor atiende un subconjunto reducido de pedidos.

El algoritmo funciona de la siguiente manera:

1. Partir desde la ubicación base del repartidor.
2. Calcular la distancia desde la posición actual hasta cada punto pendiente no visitado, usando Neo4J para obtener distancias reales.
3. Visitar el punto más cercano, marcarlo como visitado y actualizar la posición actual a esa ubicación.
4. Repetir los pasos 2 y 3 hasta haber visitado todos los puntos asignados al repartidor.
5. Acumular la distancia total recorrida en kilómetros.

Esta heurística no garantiza la solución óptima global (eso requeriría algoritmos más costosos como branch and bound o programación dinámica) pero produce resultados razonablemente buenos en tiempo lineal, lo que la hace apropiada para un sistema en tiempo real con respuesta inmediata al cliente.

7. Pipeline OLAP

La Etapa 3 incorpora una capa analítica completa sobre el sistema operacional existente. Su objetivo es separar las cargas de trabajo transaccionales de las analíticas, permitiendo ejecutar consultas agregadas sobre datos históricos sin afectar el rendimiento de la API. El pipeline sigue el patrón ETL (Extract, Transform, Load) orquestado por Apache Airflow, con Apache Hive como Data Warehouse, Apache Spark como motor de procesamiento batch y Apache Superset como herramienta de visualización.

La Figura 6 muestra el flujo completo del pipeline de datos desde el OLTP hasta los dashboards.

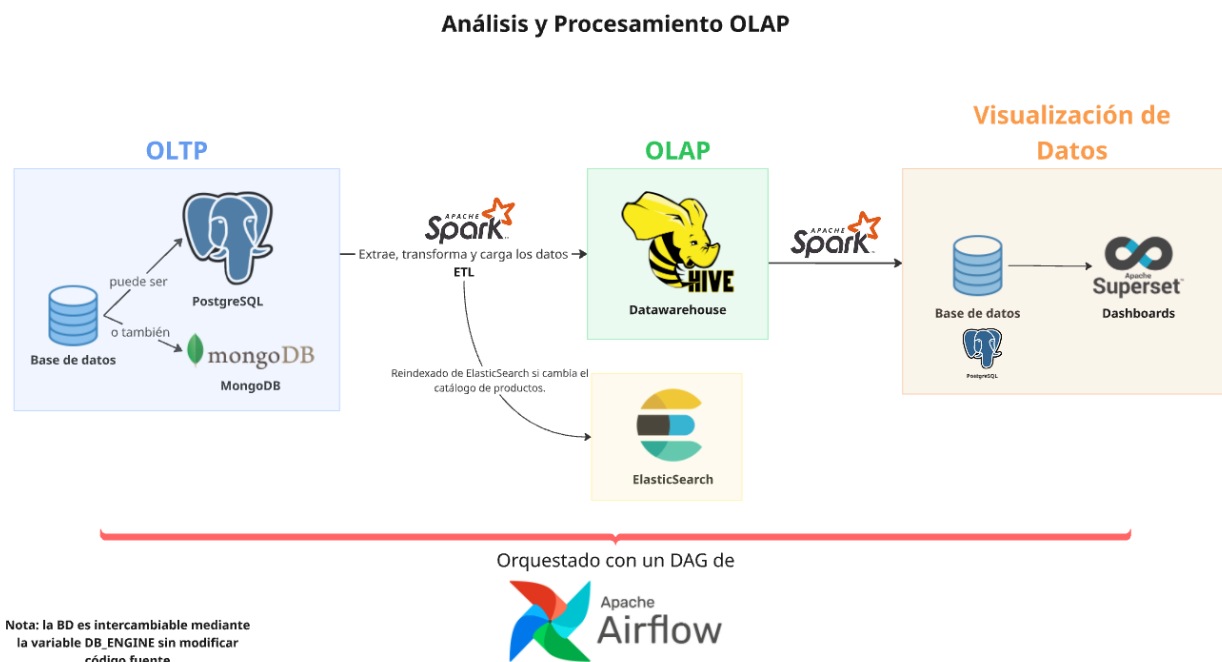


Figura 6. Diagrama explicativo de la implementación del análisis y procesamiento OLAP

El flujo de datos es el siguiente:

1. Apache Airflow dispara el pipeline diariamente a las 02:00 AM.
2. El DAG extrae datos del motor OLTP activo — PostgreSQL o MongoDB según DB_ENGINE.
3. Los datos se transforman y cargan en Apache Hive bajo un esquema estrella.

4. Apache Spark procesa el Data Warehouse y genera tablas de resultados precalculadas.
5. Apache Superset visualiza esos resultados en tres dashboards interactivos.

Una característica fundamental del diseño es que esta capa es completamente agnóstica del motor OLTP. El mismo DAG, el mismo esquema estrella en Hive y los mismos dashboards de Superset funcionan tanto con PostgreSQL como con MongoDB, cambiando únicamente la variable `DB_ENGINE` en el archivo de configuración.

7.1. Orquestación con Apache Airflow

Apache Airflow orquesta el pipeline mediante el DAG `restaurant_pipeline`, que se ejecuta automáticamente todos los días a las 02:00 AM. El DAG implementa diez tareas en secuencia, descritas en la Tabla 24.

Tarea	Tipo	Descripción
<code>branch_engine</code>	BranchPythonOperator	Lee <code>DB_ENGINE</code> y decide la rama de extracción
<code>extract_postgres</code>	PythonOperator	Extrae datos desde PostgreSQL
<code>extract_mongo</code>	PythonOperator	Extrae datos desde MongoDB (rama alternativa)
<code>join</code>	PythonOperator	Converge las dos ramas en un único flujo
<code>unify</code>	PythonOperator	Normaliza el schema de salida — idéntico independientemente del origen
<code>load_dims</code>	PythonOperator	Carga las 6 dimensiones en Hive con <code>INSERT OVERWRITE</code>
<code>load_facts</code>	PythonOperator	<code>TRUNCATE + INSERT INTO</code> en <code>fact_pedido</code> y <code>fact_reservacion</code>
<code>spark_tendencias_consumo</code>	PythonOperator	Job Spark: ingresos por mes y categoría, variación MoM

Tarea	Tipo	Descripción
spark_horarios_pico	PythonOperator	Job Spark: demanda por hora y día de semana
spark_crecimiento_mensual	PythonOperator	Job Spark: MoM, YTD y tasa de cancelación mensual
reindex_elasticsearch	PythonOperator	Reindexación del catálogo de productos al finalizar

Tabla 24. Tareas del DAG restaurant_pipeline

El DAG es idempotente por diseño: cada ejecución limpia y recarga los datos completos. Las dimensiones usan INSERT OVERWRITE, que reemplaza todo el contenido en una sola operación. Las tablas de hechos usan TRUNCATE TABLE seguido de INSERT INTO, lo que garantiza que triggerear el DAG múltiples veces produce exactamente el mismo resultado final sin duplicar datos.

Los jobs de Spark son invocados por el scheduler de Airflow mediante docker exec spark-master spark-submit. Por este motivo el scheduler necesita acceso al socket Docker (/var/run/docker.sock) montado como volumen.

7.2. Data Warehouse con Apache Hive

El Data Warehouse está implementado sobre Apache Hive 4.0.0 con un esquema estrella. Las tablas se almacenan en formato ORC con compresión Snappy y están particionadas por año y mes, lo que permite a Hive descartar particiones completas en las consultas y reducir significativamente el volumen de datos leído.

El metastore de Hive usa PostgreSQL interno (hive-metastore-db) para persistir los metadatos del schema. HiveServer2 expone el endpoint JDBC/Thrift en el puerto 10000, que usan tanto Superset como el DAG de Airflow para ejecutar queries.

7.2.1. Tablas de hechos

El schema tiene dos tablas de hechos. La Figura 7 muestra el esquema estrella de fact_pedido y la Figura 8 el de fact_reservacion.

Star schema de pedidos

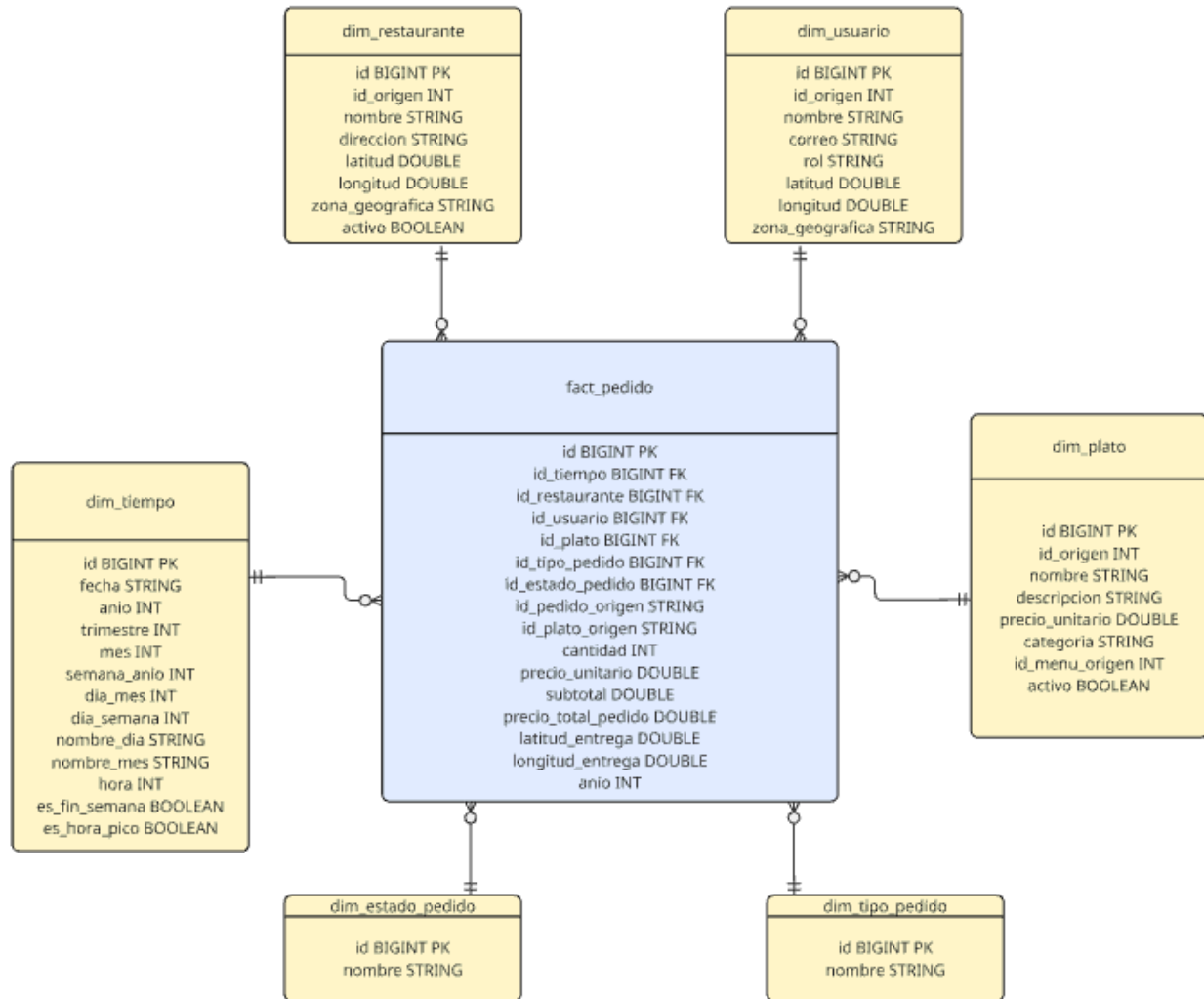


Figura 7. Diagrama del star schema para los pedidos

Star schema de reservaciones

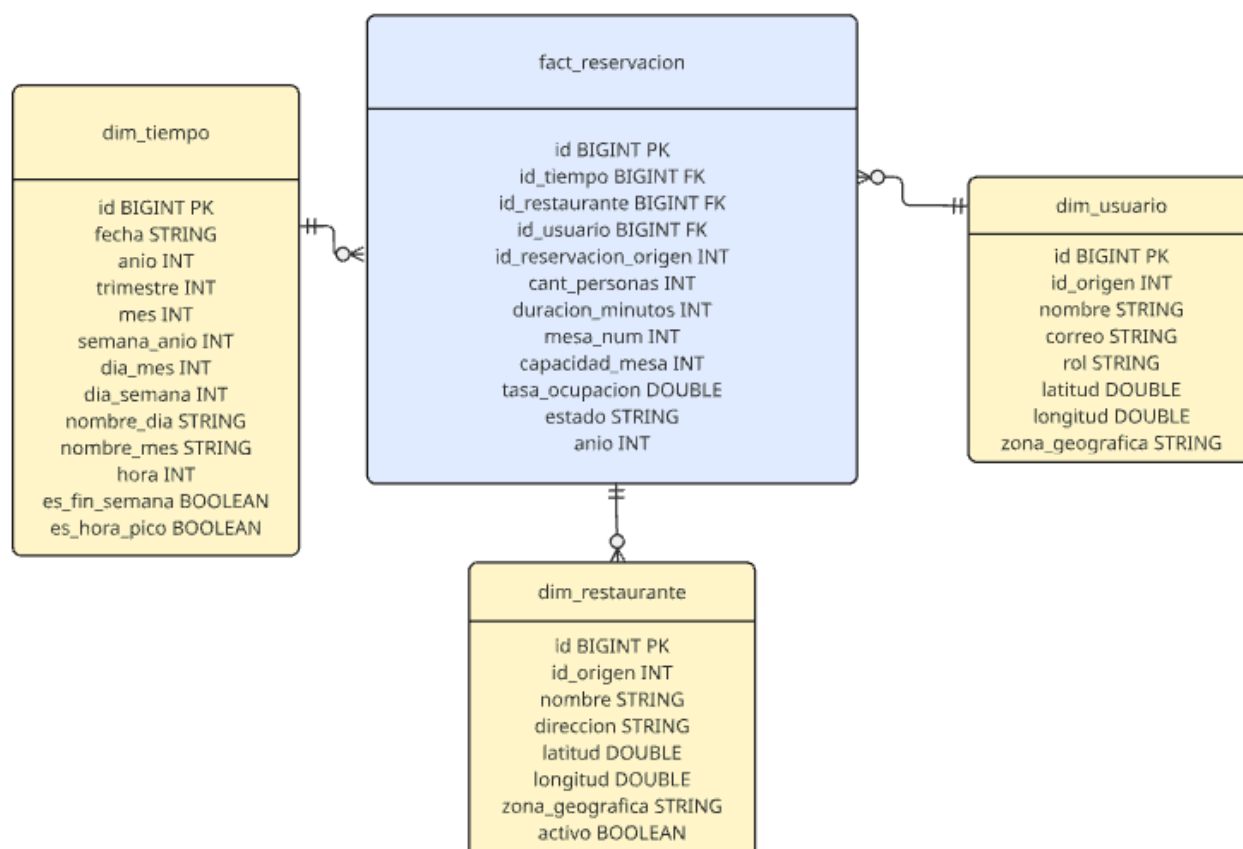


Figura 8. Diagrama del star schema para las reservaciones

Cabe destacar que `fact_pedido` almacena una fila por ítem de pedido — es decir, una fila por cada plato dentro de cada pedido. Esta granularidad permite calcular tanto métricas a nivel de pedido (usando `COUNT DISTINCT id_pedido_origen`) como métricas a nivel de producto (usando `SUM(cantidad)` o `SUM(subtotal)`). Está particionada por año y mes.

`fact_reservacion` almacena una fila por reservación, con información de ocupación de mesa y estado. También está particionada por año y mes.

Un detalle técnico importante: las columnas `id_pedido_origen` e `id_plato_origen` son de tipo `STRING` en lugar de `INT`. Esto permite que el mismo schema soporte tanto los IDs

enteros autoincrementales de PostgreSQL como los ObjectIds hexadecimales de 24 caracteres de MongoDB, sin pérdida ni truncamiento de datos.

7.2.2. Dimensiones

El schema tiene seis dimensiones que describen el contexto de cada hecho:

- `dim_tiempo` — fecha, hora, día de semana, mes, trimestre, indicadores de hora pico y fin de semana.
- `dim_restaurante` — nombre, coordenadas geográficas y zona calculada.
- `dim_usuario` — nombre, correo, rol y zona geográfica.
- `dim_plato` — nombre, categoría y precio unitario.
- `dim_tipo_pedido` — para llevar o comer aquí.
- `dim_estado_pedido` — completado o cancelado.

7.2.3. Vistas OLAP

Además de las tablas base, el schema incluye cinco vistas OLAP que permiten análisis agregados sin necesidad de escribir queries complejos:

- `v_ingresos_mes_categoria` — ingresos totales por mes y categoría de producto.
- `v_actividad_zona` — actividad de clientes y pedidos por zona geográfica.
- `v_estado_pedidos` — pedidos completados versus cancelados por mes.
- `v_horarios_pico` — distribución de demanda por hora y día de semana.
- `v_ocupacion_mesas` — tasa de ocupación de mesas por hora.

7.3. Procesamiento con Apache Spark

Apache Spark 3.5.6 ejecuta tres jobs de análisis sobre el Data Warehouse. Cada job lee las tablas de hechos y dimensiones de Hive, ejecuta transformaciones con SparkSQL y DataFrames, y guarda los resultados en tablas precalculadas dentro del mismo schema `restaurant_dw`.

Son estas tablas de resultados — no las vistas OLAP en tiempo real — las que consumen los dashboards de Superset, lo que evita timeouts por el overhead de inicialización de Tez en cada consulta.

tendencias_consumo.py calcula dos conjuntos de resultados:

- resultado_tendencias_mes_categoria — ingresos, pedidos y variación mes a mes (MoM) por mes y categoría de producto.
- resultado_top_platos_mes — top 5 platos más vendidos por mes según unidades vendidas.

horarios_pico.py calcula cuatro conjuntos de resultados:

- resultado_demanda_por_hora — total de pedidos y porcentaje del total por hora del día.
- resultado_pico_dia_semana — las tres horas de mayor demanda por cada día de la semana.
- resultado_fin_semana_vs_semana — comparativa de ticket promedio e ingresos entre días de semana y fines de semana.
- resultado_ocupacion_mesas_horaria — tasa de ocupación promedio de mesas por hora.

crecimiento_mensual.py calcula cuatro conjuntos de resultados:

- resultado_crecimiento_mensual — crecimiento MoM de pedidos, clientes únicos e ingresos por restaurante.
- resultado_crecimiento_ytd — acumulado año a la fecha de pedidos e ingresos.
- resultado_tasa_cancelacion_mensual — tasa de cancelación mensual con conteo de completados y cancelados.
- resultado_crecimiento_tipo_pedido — evolución mensual del volumen por tipo de pedido (para llevar versus en mesa).

Los tres jobs están implementados tanto como scripts de producción en `analytics/spark/jobs/` como notebooks interactivos en `analytics/spark/notebooks/`, que permiten explorar y validar los análisis paso a paso.

7.4. Visualización con Apache Superset

Apache Superset 3.1.0 presenta los resultados del pipeline mediante tres dashboards interactivos, descritos en la Tabla 25. Los dashboards están exportados como archivos YAML en `analytics/superset/dashboards/` e importables directamente en cualquier instancia de Superset 3.x.

Dashboard	Fuente de datos	Charts incluidos
Ingresos por mes y categoría	resultado_tendencias_mes_categoria	Bar chart apilado por mes y categoría, pie chart de distribución total, big number de ingreso acumulado
Actividad de clientes por zona geográfica	v_actividad_zona	Bar chart por zona cliente versus restaurante, big number de clientes únicos, tabla de desglose
Pedidos completados vs cancelados	resultado_tasa_cancelacion_mensual	Bar chart apilado por mes, pie chart de proporción global, big number de tasa de cancelación

Tabla 25. Dashboards de Apache Superset

Superset se conecta a Hive mediante el driver PyHive a través del endpoint JDBC/Thrift en el puerto 10000. Los charts que consumen las tablas resultado_* responden en menos de 2 segundos porque los datos ya están precalculados. Los charts que usan las vistas OLAP directamente pueden tardar entre 10 y 15 segundos debido al overhead de inicialización del motor de ejecución de Hive.

8. Referencia de API

El sistema expone tres grupos de endpoints distribuidos entre sus microservicios. Todos son accesibles a través de Nginx en el puerto 80, con excepción de rutas-service que también puede accederse directamente en el puerto 5050 durante desarrollo.

La documentación completa de cada endpoint — incluyendo campos de entrada, validaciones, formatos de respuesta, códigos de error y ejemplos se encuentra en el [Anexo B \(sección 12.2\)](#) de este documento.

8.1. Grupos de endpoints

8.1.1. API principal — prefijo /api/

Gestiona todas las entidades del dominio. Requiere autenticación JWT en todos los endpoints excepto registro y login.

1. **Auth** tiene los siguientes endpoints:
 - POST /api/auth/register
 - POST /api/auth/login
2. **Usuarios** tiene los siguientes endpoints:
 - GET /api/users/me
 - PUT /api/users/:id
 - DELETE /api/users/:id
3. **Restaurantes** tiene los siguientes endpoints:
 - POST /api/restaurants
 - GET /api/restaurants
4. **Menús** tiene los siguientes endpoints:
 - POST /api/menus
 - GET /api/menus/:id
 - PUT /api/menus/:id
 - DELETE /api/menus/:id
5. **Reservaciones** tiene los siguientes endpoints:
 - POST /api/reservations
 - DELETE /api/reservations/:id
6. **Pedidos** tiene los siguientes endpoints:
 - POST /api/orders
 - GET /api/orders/:id

8.1.2. Microservicio de búsqueda — prefijo /search/

No requiere autenticación. Usa Elasticsearch como motor de índice. Se tienen los siguientes endpoints:

1. GET /search/products?q=texto: es para búsqueda textual en nombre, categoría y descripción
2. GET /search/products/category/:cat: es para filtrar por categoría exacta
3. POST /search/reindex: es para reindexar el catálogo completo en Elasticsearch

8.1.3. Microservicio de grafos y rutas — prefijos /grafos/ y /rutas/

No requiere autenticación. Se conecta a Neo4J y a la base de datos operacional activa. Se tienen los siguientes endpoints:

1. GET /grafos/co-compras: los 5 pares de platos más comprados juntos
2. GET /grafos/usuarios-influyentes: usuarios ordenados por cantidad de recomendaciones
3. GET /grafos/camino-minimo?origen=X&destino=Y: camino mínimo entre dos zonas de Cartago
4. POST /rutas/asignar: asignar pedidos a repartidores con ruta optimizada

8.2. Autenticación

Los endpoints protegidos requieren el header:

```
Authorization: Bearer <access_token>
```

El token se obtiene desde Keycloak mediante POST /api/auth/login y tiene una duración de 300 segundos. Para el detalle completo del flujo de autenticación y todos los campos de cada endpoint, referirse al [Anexo B](#).

Parte III. Operación

9. Comandos de operación

Esta sección centraliza todos los comandos necesarios para operar el sistema en sus distintos modos de despliegue. Los comandos asumen que el directorio de trabajo es la raíz del repositorio (Proyecto2-BasesII/) y que Docker Desktop está corriendo con al menos 8 GB de RAM asignados.

9.1. Docker Compose

A continuación, se presentan los comandos para levantar los diferentes stacks.

9.1.1. Stack principal con PostgreSQL es:

```
# Levantar el stack completo con PostgreSQL
```

```
docker compose --project-directory . \  
    -f deploy/local/docker/docker-compose.postgres.yml \  
    up -d
```

```
# Detener el stack
```

```
docker compose --project-directory . \  
    -f deploy/local/docker/docker-compose.postgres.yml \  
    down
```

9.1.2. Stack principal con MongoDB

```
# Levantar el stack completo con MongoDB sharded

docker compose --project-directory . \
    -f deploy/local/docker/docker-compose.mongo.yml \
    up -d


# Verificar que mongo-init completó el sharding antes de
continuar

docker logs mongo-init --tail 5

# Debe mostrar: === mongo-init completado ===


# Detener el stack

docker compose --project-directory . \
    -f deploy/local/docker/docker-compose.mongo.yml \
    down
```

9.1.3. Stack analítico OLAP

Copiar el driver JDBC de PostgreSQL al volumen de Hive
(necesario una vez)

```
docker run --rm `
                                                    -v
"${PWD}/analytics/hive/drivers/postgresql.jar:/source/postgresql
.jar" `
    -v "proyecto2-basesii_hive_warehouse_data:/dest" `
    alpine sh -c "cp /source/postgresql.jar /dest/postgresql.jar
&& chmod 777 /dest"
```

Levantar el stack de analytics

```
docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.analytics.yml `
up -d
```

Inicializar el star schema de Hive (solo la primera vez o tras reinicio limpio)

```
docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.analytics.yml `
run --rm hive-init
```

Detener el stack de analytics

```
docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.analytics.yml `
down
```


9.1.4. Stack de Neo4J y rutas-service

```
# Levantar Neo4J y rutas-service

docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.neo4j.yml `
up -d neo4j rutas-service


# Cargar ubicaciones de Cartago en Neo4J (solo la primera vez)

docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.neo4j.yml `
exec neo4j cypher-shell -u neo4j -p password123 `
    -f /var/lib/neo4j/import/ubicaciones.cypher


# Migrar datos del OLTP a Neo4J

docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.neo4j.yml `
run neo4j-migrate


# Detener Neo4J y rutas-service

docker compose --project-directory . `
    -f deploy/local/docker/docker-compose.neo4j.yml `
down
```

9.1.5. Cambiar entre motores de base de datos

1. Pausar el DAG para evitar ejecuciones automáticas mientras se cambia

```
docker exec airflow-scheduler airflow dags pause
restaurant_pipeline
```

2. Limpiar variables de sesión de PowerShell que puedan interferir

```
Remove-Item Env:MONGO_URL -ErrorAction SilentlyContinue
```

```
Remove-Item Env:DB_ENGINE -ErrorAction SilentlyContinue
```

3. Editar .env: cambiar DB_ENGINE=postgres <->
DB_ENGINE=mongodb

4. Bajar el stack actual y levantar el nuevo

```
docker compose --project-directory . `
  -f deploy/local/docker/docker-compose.postgres.yml `
  down # o mongo.yml según corresponda
```

```
docker compose --project-directory . `
  -f deploy/local/docker/docker-compose.mongo.yml `
  up -d # o postgres.yml según corresponda
```

5. Reiniciar el scheduler para que tome el nuevo DB_ENGINE

```
docker rm -f airflow-scheduler
```

```
docker compose --project-directory . `
  -f deploy/local/docker/docker-compose.analytics.yml `
```

```
up airflow-scheduler -d
```

```
# 6. Despausar el DAG y triggerear una corrida
```

```
docker      exec      airflow-scheduler      airflow      dags      unpause  
restaurant_pipeline
```

```
docker      exec      airflow-scheduler      airflow      dags      trigger  
restaurant_pipeline
```

9.1.6. Gestión del DAG de Airflow

```
# Pausar el DAG (recomendado durante desarrollo)
```

```
docker      exec      airflow-scheduler      airflow      dags      pause  
restaurant_pipeline
```

```
# Despausar el DAG
```

```
docker      exec      airflow-scheduler      airflow      dags      unpause  
restaurant_pipeline
```

```
# Triggerear manualmente una corrida
```

```
docker      exec      airflow-scheduler      airflow      dags      trigger  
restaurant_pipeline
```

```
# Ver historial de runs exitosos
```

```
docker exec airflow-scheduler airflow dags list-runs `  
-d restaurant_pipeline --state success
```

9.1.7. Generación de datos de prueba

Con PostgreSQL

```
$env:DB_ENGINE = "postgres"
```

```
python db/seed/generate_test_data.py
```

Con MongoDB

```
$env:DB_ENGINE = "mongodb"
```

```
$env:MONGO_URL = "mongodb://localhost:27017/"
```

```
$env:MONGO_DB = "restaurantdb"
```

```
python db/seed/generate_test_data.py
```

Limpiar datos anteriores antes de insertar (RESET)

```
$env:RESET = "true"
```

```
python db/seed/generate_test_data.py
```

9.1.8. Verificación de integridad del Data Warehouse

Ejecutar en Superset SQL Lab (<http://localhost:8089>) o via beeline:

```
SELECT 'fact_pedido' AS tabla, COUNT(*)  
AS filas FROM fact_pedido  
  
UNION ALL SELECT 'fact_reservacion', COUNT(*)  
FROM fact_reservacion  
  
UNION ALL SELECT 'resultado_tendencias_mes_categoria', COUNT(*)  
FROM resultado_tendencias_mes_categoria  
  
UNION ALL SELECT 'resultado_tasa_cancelacion_mensual', COUNT(*)  
FROM resultado_tasa_cancelacion_mensual
```

```
UNION ALL SELECT 'resultado_demanda_por_hora',          COUNT(*)
FROM resultado_demanda_por_hora
```

```
UNION ALL SELECT 'resultado_crecimiento_mensual',      COUNT(*)
FROM resultado_crecimiento_mensual;
```

9.1.9. URLs de acceso — Docker Compose

Refiérase a la Tabla 26 para ver un resumen de las URLs de acceso al hacer el deployment con Docker Compose.

Servicio	URL	Credenciales
API REST	http://localhost:3000	JWT via Keycloak
Nginx	http://localhost:80	—
Keycloak	http://localhost:8080	admin / admin
Neo4J Browser	http://localhost:7474	neo4j / password123
rutas-service	http://localhost:5050	—
Airflow UI	http://localhost:8090	admin / admin
Hive Web UI	http://localhost:10002	—
Spark Master UI	http://localhost:8081	—
Superset	http://localhost:8089	admin / admin

Tabla 26. URLs de acceso en Docker Compose

9.2. Kubernetes

Acto seguido, se adjuntan los comandos para hacer el deployment con Kubernetes.

9.2.1. Primer despliegue

```
# Construir las imágenes locales
docker build -t restaurantes/api:latest ./api
docker build -t restaurantes/search:latest ./search-service
docker build -t restaurantes/rutas-service:latest
./rutas-service

# Despliegue completo desde cero
cd deploy/local/k8s
.\deploy.ps1 -Clean
```

Importante: el flag -Clean elimina el cluster anterior. Usarlo solo al empezar de cero.

9.2.2. Despliegue incremental

```
# Sin borrar el cluster existente
.\deploy.ps1

# Stack de analytics (Hive, Spark, Airflow, Superset)
.\apply-analytics.ps1

# Neo4J y rutas-service
.\apply-neo4j.ps1

# Neo4J y rutas-service con MongoDB
.\apply-neo4j.ps1 -Engine mongo
```

9.2.3. Acceso a servicios en Kubernetes

```
# Abrir el tunnel de Minikube (mantener abierto en terminal  
separada)
```

```
minikube tunnel
```

```
# Port-forwards para servicios internos
```

```
kubect1 port-forward svc/keycloak-service 9999:8080 -n  
restaurantes
```

```
kubect1 port-forward svc/neo4j-service 7474:7474 -n  
restaurantes
```

```
kubect1 port-forward svc/neo4j-service 7687:7687 -n  
restaurantes
```

```
kubect1 port-forward svc/airflow-webserver 8090:8080 -n  
restaurantes
```

```
kubect1 port-forward svc/superset 8089:8088 -n  
restaurantes
```

```
kubect1 port-forward svc/hive-server 10000:10000 -n  
restaurantes
```

9.2.4. Escalar servicios

```
# Escalar la API a 5 instancias
```

```
kubect1 scale deployment api --replicas=5 -n restaurantes
```

```
# Escalar el microservicio de búsqueda a 4 instancias
```

```
kubect1 scale deployment search --replicas=4 -n restaurantes
```

```
# Ver estado de los pods
```

```
kubect1 get pods -n restaurantes
```

```
# Ver HPA (auto-scaling configurado)
```

```
kubect1 get hpa -n restaurantes
```

9.2.5. Cambiar motor de base de datos en Kubernetes

```
# Editar el ConfigMap activo
kubectl edit configmap app-config-postgres -n restaurantes
# Cambiar DB_ENGINE: "postgres" -> "mongodb"

# Reiniciar los pods para que tomen el nuevo valor
kubectl rollout restart deployment/api -n restaurantes
kubectl rollout restart deployment/search -n restaurantes
```

9.2.6. Limpiar todo

```
minikube delete
```

9.3. CI/CD

El sistema cuenta con un pipeline de integración y entrega continua implementado con GitHub Actions que se ejecuta automáticamente en cada push a cualquier rama. La Tabla 26 describe los tres jobs del pipeline.

Job	Condición	Descripción
Pruebas Unitarias	Siempre	Ejecuta todos los tests en tests/unit/ con cobertura mínima del 90%
Pruebas integración	Job 1 exitoso	Levanta PostgreSQL y Redis como servicios, ejecuta tests/integration/
Build y Push Docker	Solo rama main	Construye la imagen Docker y la publica en GitHub Container Registry (ghcr.io)

Tabla 26. Jobs del pipeline CI/CD

10. Decisiones de diseño

Esta sección documenta las decisiones arquitectónicas y de implementación más relevantes del sistema, explicando el razonamiento detrás de cada una. El objetivo es que cualquier desarrollador que se incorpore al proyecto pueda entender por qué el sistema está construido como está, y no solo cómo funciona.

10.1. Arquitectura y separación de responsabilidades

A continuación, se explican las decisiones con respecto a la arquitectura y la delegación de responsabilidades implementada en el proyecto.

- **Patrón DAO con DAOFactory**

Permite cambiar el motor de base de datos mediante una variable de entorno sin modificar ningún controlador, servicio ni ruta. El único punto del sistema donde se evalúa `DB_ENGINE` es `DAOFactory.js`. Los servicios reciben el DAO ya instanciado sin saber si viene de PostgreSQL o MongoDB, lo que cumple el principio DIP de SOLID y facilita las pruebas unitarias con mocks.

- **Microservicio de búsqueda separado**

ElasticSearch tiene requisitos de recursos distintos a la API CRUD — consume más memoria y CPU por sus índices invertidos. Separarlo en `search-service` permite escalar ambos componentes de forma independiente y mantiene la responsabilidad única por servicio. La API CRUD nunca toca ElasticSearch directamente.

- **Un solo microservicio para grafos y rutas**

Los puntos 5 y 6 del enunciado comparten la misma fuente de datos (Neo4J) y una lógica relacionada. Siguiendo el mismo principio con el que se separó `search-service` de la API principal, se creó `rutas-service` como microservicio independiente que cubre ambas responsabilidades. Esto evita añadir lógica de grafos a la API CRUD, que ya tiene suficientes responsabilidades.

- **Conexión directa a la BD en rutas-service sin patrón DAO**

A diferencia de la API principal, `rutas-service` no implementa el patrón DAO completo. Solo necesita una operación de lectura simple sobre pedidos pendientes — no CRUD completo. Una conexión directa con pg y mongodb es más simple y suficiente para este caso de uso, evitando sobreingeniería en un microservicio de análisis.

- **Archivo docker-compose separado para Neo4J y analytics**

Neo4J, rutas-service y el stack analítico son componentes que se levantan cuando se necesitan, no permanentemente en producción. Mantenerlos en archivos Compose separados (docker-compose.neo4j.yml y docker-compose.analytics.yml) evita que el stack principal se vuelva más pesado y permite levantar cada capa de forma independiente. Ambos comparten la red del stack principal mediante una red externa declarada.

10.2. Base de datos y persistencia

Aquí se explican las decisiones con respecto a la persistencia de datos y BD.

- **Replica Set MongoDB**

Se configuran 3 nodos (1 primario + 2 secundarios) para garantizar alta disponibilidad. Si el primario falla, un secundario toma el control automáticamente sin intervención manual. El contenedor mongo-init configura el replica set y el sharding al primer arranque usando MERGE para garantizar idempotencia.

- **MERGE en lugar de CREATE en Neo4J**

El script neo4j/migrate.js usa MERGE para todas las operaciones de creación en Neo4J. Esto garantiza que el script se pueda correr múltiples veces sin duplicar datos, lo que facilita re-migraciones cuando cambian los datos de la base de datos principal.

- **Geolocalización simulada**

Como el sistema no tiene un módulo de geocodificación real de direcciones de clientes, se simula asignando una de las 8 ubicaciones reales de Cartago a cada pedido. Las ubicaciones se almacenan en Neo4J como nodos Ubicacion con coordenadas lat/long reales y conexiones con distancias reales entre ellas, cumpliendo el requisito de geolocalización del enunciado sin necesidad de integrar un servicio externo de mapas.

- **id_pedido_origen e id_plato_origen como STRING en Hive**

Las columnas de origen en fact_pedido son de tipo STRING en lugar de INT. PostgreSQL usa IDs enteros autoincrementales, pero MongoDB usa ObjectId hexadecimales de 24 caracteres que no caben en un INT de 32 bits — Hive los truncaría silenciosamente a NULL. Usar STRING permite que el mismo schema soporte ambos motores sin pérdida de datos.

10.3. Rendimiento y escalabilidad

Siguiente, están las decisiones que conciernen a rendimiento y escalabilidad.

- **Nginx como único punto de entrada**

Oculto la topología interna de contenedores al cliente externo, permite escalar la API horizontalmente de forma transparente con round-robin, y centraliza el enrutamiento entre todos los microservicios. El cliente siempre habla con el puerto 80, sin importar cuántas instancias de la API estén corriendo detrás.

- **Cache-Aside con Redis**

El servicio primero busca en Redis, luego consulta la BD si no hay hit, y guarda el resultado para futuras consultas. La invalidación es por evento mediante deletePattern al modificar datos, no por TTL solamente. Redis es compartido por todas las instancias de la API, lo que garantiza consistencia del caché cuando la API está escalada horizontalmente.

- **Tablas resultado_* precalculadas por Spark**

Los dashboards de Superset consumen las tablas resultado_* generadas por Spark en lugar de las vistas OLAP en tiempo real. Esto es porque Hive con el motor Tez tiene un overhead de inicialización de entre 10 y 15 segundos por query, incluso para tablas pequeñas. Las tablas precalculadas responden en menos de 2 segundos porque no requieren planificación de jobs distribuidos.

10.4. Orquestación y operación

Aquí se presentan las decisiones que conciernen a orquestación y operación.

- **Idempotencia del DAG de Airflow**

Cada corrida del DAG limpia y recarga los datos completos en lugar de hacer cargas incrementales. Las dimensiones usan INSERT OVERWRITE y las tablas de hechos usan TRUNCATE + INSERT INTO. Esto simplifica significativamente el manejo de errores — si el DAG falla a mitad, basta con triggerarlo de nuevo desde el inicio sin necesidad de limpiar manualmente el estado parcial.

- **Inyección de dependencias en servicios y controladores**

Los servicios reciben el DAO por constructor y los controladores reciben el servicio. Esto facilita las pruebas unitarias con mocks y cumple el principio DIP de SOLID. Ningún controlador instancia directamente sus dependencias.

- **Hive Metastore con PostgreSQL dedicado**

El metastore de Hive usa su propia instancia de PostgreSQL (hive-metastore-db) aislada de la base de datos de la aplicación y de la de Keycloak. Esto evita interferencias entre los sistemas y permite que cada base de datos sea respaldada, escalada o migrada de forma independiente.

11. Tecnologías utilizadas

En esta sección se presenta la Tabla 27 la cual resume todas las tecnologías del sistema con su versión y rol específico.

Tecnología	Versión	Rol en el sistema
Node.js	20 LTS	Runtime del servidor para API y search-service
Express	5.x	Framework HTTP para ambos microservicios
PostgreSQL	16	Base de datos relacional principal
MongoDB	7	Base de datos documental alternativa
ElasticSearch	8.x	Motor de búsqueda textual
Redis	7	Caché distribuida en memoria
Keycloak	24.0	Servidor de identidad y autenticación OAuth2/OIDC
Nginx	Alpine	Proxy inverso y balanceador de carga
Docker	24+	Contenedorización de todos los servicios
Docker Compose	v2	Orquestación local de contenedores
Github Actions	-	Pipeline CI/CD automatizado
Jest	29	Framework de pruebas unitarias e integración
Neo4j	5.18	Base de datos de grafos para análisis de relaciones y rutas
neo4j-driver	5.x	Driver oficial Node.js para conectar a Neo4J via Bolt
APOC	plugin	Utilidades de Neo4J para importar y exportar datos

Tecnología	Versión	Rol en el sistema
Graph Data Science (GDS)	plugin	Algoritmos de grafos: Dijkstra para caminos mínimos
Apache Hive		Data Warehouse para esquema estrella OLAP
Apache Spark		Procesamiento de grandes volúmenes de datos
Apache Airflow		Orquestación del pipeline de datos
Metabase / Superset		Visualización de dashboards analíticos

Tabla 27. Resumen de las tecnologías utilizadas en el proyecto

12. Anexos

A fin de complementar este documento, se tienen tres anexos: el video demo de la tercera etapa, la documentación completa de la API y el video demo de la segunda etapa.

12.1. Anexo A: video demo (tercera etapa)

<https://youtu.be/6CSnCE1wCcg>

12.2. Anexo B: documentación completa de API

El documento se encuentra en la carpeta docs del repositorio oficial del proyecto bajo el nombre **Anexo Referencia API.pdf**.

12.3. Anexo C: video demo (segunda etapa)

<https://youtu.be/vNMVTh52BFw>